

Evidence-Backed Authority Verification for Autonomous Agents: Measuring and Governing Root-Equivalent Execution Paths

Status: submission draft, revision 3. Single-host case study ($n = 1$). Every quantitative claim in §13 is reproducible from the artifacts named in §15 and was produced by the runs recorded in ``run_history.jsonl``. External references support mechanism descriptions and prior work only; no citation is used as evidence for a property of the evaluated host.

Reader's guide

This page is orientation, not argument. It states the result in ordinary language and defines the terms. Nothing here is a claim the rest of the paper does not make; readers who want the argument can skip to the Abstract.

The question, in plain terms

A software agent runs on a machine as an ordinary user account. A protected set of records on that machine is owned by a separate account, and the rule is that only that account may change the records — the agent has to ask it. **We tried to prove that the agent has no other way to change those records.**

The answer

We could not prove it, and the system says so rather than guessing.

The agent's account belongs to a group that grants it access to the Docker service. Docker runs as the machine's administrator account, so anything the agent asks Docker to do is done with administrator power — including changing the protected records. Nothing here is a break-in: the machine was configured to grant that access, and every permission check involved *passes*. That is why no sandbox, filter or hardening setting fixes it. The only fix is to stop granting the access, and only a human administrator can do that.

Eighteen further possibilities could not be settled at all, because deciding them means reading files the agent is not permitted to read. The system treats "could not settle" as a blocking answer, not as a clean result.

So the final answer is **blocked**, with the reason named. That is the intended behaviour: the system is built to refuse to certify anything it has not measured.

The part we think is worth reading

The system found three faults **in itself**, and each has the same shape: *the right fact was measured and written down, but the decision looked at a different fact.*

1. The scan for dangerous programs looked for one marker and missed a second, entirely different one. Eleven programs were therefore invisible to it.
2. A fingerprint meant to prove that repeated checks measured the same thing did not actually cover what was measured. Adding eleven items to the list left the fingerprint unchanged.
3. When the measuring program was itself run inside a restricted sandbox, it reported a perfectly clean machine — on a machine that had not changed.

None was found by an attacker. All three were found by the system's own checks.

Where to start

If you are...	Read
Deciding whether to deploy	This page, then §13.2 (the verdict) and §16 (limitations)
Reviewing the security argument	§3 (problem), §4 (threat model), §12 (findings), §14 (threats and reviewer questions)
Checking the formal claims	§9 in full — the theorems, their proofs, and §9.6 on where the model and the code differ
Reproducing the numbers	§15, including Table 16, which maps every number to the file and field it came from
Looking for prior work	§2

Terms used throughout

Term	Meaning
Authority principal	The account allowed to change the protected records. Everyone else must ask it
Authority carrier	Any mechanism by which an account can gain power beyond its own. Not necessarily a flaw — a deliberate grant is still a carrier, and is the worst kind, because there is no flaw to fix
Root-equivalent	Confers the machine administrator's power, by whatever route
Surface	One category of mechanism that can carry authority — group memberships, setuid programs, file capabilities, and so on. Seven are modelled; §16.1 lists what is not
Path	One specific instance of a surface, e.g. one particular program
Discriminator	The specific fact, read from a specific place, that settles what a path can do. No classification is made without one
Measurement context	The identity and restrictions the measuring program itself ran under. Recorded, because it changes what the measurement can see
Host-representative	A measurement context that matches the agent's real one. Measurements taken any other way are not accepted as evidence about the machine
Unresolved	The mechanism exists and the deciding fact could not be obtained. Always blocks
Coverage	Which surfaces a measurement actually contains. Computed from its contents, never from its own claim about itself
Evidence digest	A fingerprint over the security-relevant result, used to detect when something changed between runs
Fails closed	When information is missing, the answer is "no" rather than "probably fine"

The four possible answers, worst first: `BLOCKED_ROOT_EQUIVALENCE` (a route to administrator power was measured), `BLOCKED_AUTHORITY_EQUIVALENCE` (a route to the protected account, or an unfinished analysis), `BLOCKED_PRIVILEGE_UNCERTAIN` (something could not be settled), and `VERIFIED_EXCLUSIVE_AUTHORITY` — the only positive answer, and one this evaluation did not produce.

Contents

Reader's guide	1
The question, in plain terms	1
The answer	1
The part we think is worth reading	1
Where to start	2
Terms used throughout	2
Abstract	4
1. Introduction	5
1.1 Research gap	5
1.2 Contributions	6
1.3 What this paper does not claim	7
2. Related work	7
3. Problem statement	8
4. Threat model	9
5. System architecture	9
6. Authority surface model	10
6.1 Definitions	10
6.2 The three-valued distinction	10
6.3 Surfaces	12
7. Measurement methodology	12
7.1 Read-only discipline	13
7.2 Measurement context binding	13
7.3 Coverage: derived, not declared	13
7.4 Discriminators	14
8. Classification model	15
8.1 Terminal states	15
8.2 Verdict function	15
9. Formal authority exclusivity model	16
9.1 Objects	16
9.2 Measurement and classification	17
9.3 The verdict predicate	17
9.4 Safety properties	18
9.5 What is not true: the labels are not a total order	19
9.6 Fidelity of the formalisation	20
10. Evidence integrity model	21
11. Implementation	22
12. Findings	23
12.1 The residual carriers	23
12.2 Finding 1 — inventory completeness failure	24
12.3 Finding 2 — evidence reproducibility failure	24
12.4 Finding 3 — measurement context (prior pass, summarised)	25
12.5 The common structure	25
13. Security evaluation	26
13.1 Measured authority topology	26
13.2 Verdict	26
13.3 Validation	27

14. Threats to validity and expected reviewer questions	28
14.1 Threats to validity	28
14.2 Expected reviewer questions	28
14.3 Proposed independent adversarial evaluation	29
15. Reproducibility	30
16. Limitations	32
16.1 The surface list is curated	32
16.2 Operator evidence cannot be closed by the agent	33
16.3 Verifier integrity boundary	33
16.4 Scope of the security claim	33
17. Future work	33
17.1 Authority Surface Registry	33
17.2 Verifier Integrity Layer	34
17.3 Other directions	34
18. Conclusion	34
19. References	35

Abstract

Autonomous coding agents execute with the ambient authority of the account that launched them, and the governance layers built over them routinely assert that privileged actions are impossible for the agent. That assertion is normally derived from the absence of an observation rather than from a measurement. **Absence of observation is not equivalent to measured absence**, and a verifier that conflates them will certify a host it has not measured. Conventional privilege auditing does not close this gap: it enumerates candidate escalation paths and reports them to a human, so a check that found nothing and a check that could not run are rendered identically and the reader supplies the missing judgement.

We report an evidence-backed authority verification system for a canonical-state promotion authority, in which a governed store may be mutated only through an authority principal and the agent must be provably unable to assume that principal. The system verifies 19 conditions over a measured privilege topology and refuses to certify exclusivity unless every enumerated authority carrier reaches a terminal classification and every inventory proves it was collected from a host-representative process context. Its reasoning is three-valued — *measured absence*, *unresolved possibility*, *proven impossibility* — and only the first and third are compatible with a positive verdict; unresolved possibility blocks unconditionally. We formalise the verdict function and prove that no unresolved path, no unmeasured required surface, and no context-invalid inventory can yield a positive verdict. We also prove the corresponding negative result: the four verdict labels are not a severity total order, and we give both reachable counterexamples.

The system's central claim is a constraint on verifier construction: *a recorded fact is insufficient if a different fact is consumed by the verifier*. We report three failures of this form in our own system, each found by its own gates rather than by an external reviewer. **Finding 1, inventory completeness:** setuid enumeration is structurally incapable of discovering Linux file capabilities, and 11 capability-bearing binaries were absent from the authority model entirely. **Finding 2, digest completeness:** an evidence digest stable across four consecutive runs demonstrated determinism but not measurement completeness, because the digest did not cover the privilege inventory — adding an entire surface left it bit-identical. **Finding 3, context binding:** an identical collector reported a clean host from inside a restricted process context, with no change to the host at all.

On the evaluated host the final verdict is `BLOCKED_ROOT_EQUIVALENCE`, specific reason `BLOCKED_ROOT_EQUIVALENCE_DOCKER`, with `authority_exclusivity_proven: false`. Two measured root-equivalent carriers remain — membership of the `docker` group and the resulting write access to the rootful Docker socket — and 18 further paths are unresolved because the artifacts that would decide them are unreadable without privileged access.

We claim neither complete Linux privilege discovery, nor elimination of root-equivalence, nor universal authority closure, and the blocked verdict is not a certification of any kind. The contribution is an architecture that fails closed when authority exclusivity cannot be proven, and that makes the boundary between measured and unmeasured explicit and machine-checkable.

1. Introduction

A governance layer for an autonomous agent makes claims of the form "the agent cannot perform action X ". In deployed systems those claims are justified in one of three ways: the agent has no code path to X ; an access control check denies X ; or an adversarial test suite attempted X and failed. All three are arguments about what was observed. None establishes that no *unobserved* path to X exists. Program analysis has long acknowledged this gap explicitly — the soundness manifesto [Liv15] argues that analyses should declare which constructs they deliberately do not handle rather than imply exhaustiveness. The same discipline is absent from most deployment-time authority claims.

The gap is not academic. In the system studied here, an earlier version of the verifier computed its exclusivity condition as `not bool(root_equivalent_paths)`. A companion field, `unknown_paths`, recorded mechanisms that had been found but not resolved — and was consumed by no condition. A host on which the only remaining escalation path was unresolved would have been certified `VERIFIED_EXCLUSIVE_AUTHORITY`. The defect was not in the access control design, the threat model, or the attack suite. It was in the treatment of unresolved state as equivalent to absent state.

Agent-layer security research has concentrated on the input channel — indirect prompt injection [GAM+23], benchmark environments for injection attacks and defences [Deb24], emulated-sandbox risk identification [Rua24], and design-level containment of untrusted data flows [Deb25]. That work asks whether an agent can be induced to *attempt* a harmful action. This paper asks the orthogonal question: given that the agent will attempt it, what authority does the host actually confer on the account it runs as, and can that be *measured* rather than assumed. The two questions compose; neither substitutes for the other.

1.1 Research gap

Absence of observation is not equivalent to measured absence.

The two are routinely represented by the same value — an empty result set — and consumed by the same decision. A verifier that cannot distinguish them has, in effect, one predicate where it needs two: it can report *nothing was found*, but not whether that is because nothing is there, because the enumerator cannot see the mechanism, or because the deciding fact was unreadable. Every failure reported in this paper is an instance of that missing distinction.

Closing the gap requires three things that a report-producing tool does not need: a representation in which "unresolved" is a first-class state rather than an empty list; a verdict function in which that state is load-bearing;

and evidence binding strong enough that the coverage of a measurement cannot be asserted by the measurement itself.

Distinction from conventional privilege auditing. Established auditors — LinPEAS, Lynis, OpenSCAP [pe-ass, lynis, oscan] — enumerate escalation surfaces far more broadly than the seven surfaces modelled here, and this work does not compete with them on enumeration breadth (§16.1 concedes the gap in their favour). Three properties separate the two activities:

	Conventional privilege auditing	Authority verification (this work)
Output	A ranked report; a human decides what it means	A machine-consumed verdict that gates a deployment decision
Treatment of silence	"Found nothing" and "could not check" render identically	Distinct states; the second blocks unconditionally (§6.2)
Coverage	Asserted by the tool's design	Derived from the inventory's own contents; a tool that measured nothing cannot claim to have measured everything (§7.3)
Measurement context	Implicit — whatever context the tool ran in	Recorded, fingerprinted, and an admissibility precondition (§7.2)
Failure mode of interest	A missed escalation path	A <i>certification</i> issued over an unmeasured surface

The distinction is not that auditors are unsound and this system is sound. Both are incomplete, and §16.1 says so. The distinction is what each does with its own incompleteness: an auditor externalises it to the reader, and a verifier must represent it internally or silently discard it.

This paper describes the resulting system, its measurement methodology, a formal model of its verdict function, and — at greater length than is conventional — the failures of the same class that the system's own gates detected in itself.

1.2 Contributions

C1 — A three-valued authority model. *Measured absence*, *unresolved possibility* and *proven impossibility* are represented distinctly, and unresolved possibility blocks unconditionally. We give the argument for why the blocking must be unconditional rather than risk-weighted, and the three mechanisms that keep the middle state from decaying into the first (§6.2, §8).

C2 — A formal model with a proved safety property, and a proved negative result. No unresolved path, no unmeasured required surface, no context-invalid inventory, and no added root-equivalent carrier can yield `VERIFIED_EXCLUSIVE_AUTHORITY` (Theorem 1, Corollary 1). Separately, the four verdict labels are *not* a severity total order, because one label occupies two precedence positions; we give both reachable counterexamples and the operational consequence — an adversary who deletes evidence cannot manufacture a certification but can degrade a specific blocker into a generic one (Theorem 2, §9).

C3 — Measurement context binding. An identical collector reports a clean privilege inventory from inside a restricted process context, with no change to the host; a control measurement establishes that the host did not change. We give the admissibility predicate that makes such an inventory *inadmissible* rather than merely suspect, and show why an inadmissible inventory must be neutralised rather than discarded — discarding it would delete its root-equivalent findings and improve the verdict (§7.2, Finding 3).

C4 — The recorded-versus-consumed failure class, with three instances in our own system. A recorded fact is insufficient if a different fact is consumed by the verifier. Finding 1 (inventory completeness): setuid enumeration cannot discover file capabilities, and an entire surface was absent while coverage reported

complete. Finding 2 (digest completeness): a stable digest across four runs proved determinism, not measurement completeness. Finding 3 (context binding): a sandbox produced false cleanliness. Each is accompanied by the remedy that makes the consumed set derived from, and provably equal to, the recorded set (§10, §12).

C5 — The claim that verification systems are themselves governed components. All three findings were produced by the system's gates rather than by an external reviewer, which is evidence that the discipline yields findings when turned inward. We state the verifier integrity boundary explicitly, including two gaps that remain open at submission time (§16.3).

C6 — An evaluation on a real host terminating in a blocked verdict, with the residual authority carriers enumerated and attributed, the unresolved remainder each paired with the exact privileged read that would decide it, and a positive control establishing that the gate is capable of emitting a non-blocked result (§13).

1.3 What this paper does not claim

- Not complete discovery of Linux privilege-granting mechanisms. `REQUIRED_SURFACES` is a curated list; §16.1 names known omissions, and Finding 1 is empirical proof that such a list can be wrong.
- Not that the evaluated host is secure. It claims the opposite, with evidence.
- Not that root-equivalence was eliminated. Two carriers remain measured and present.
- Not universal authority closure. Closure was not reached; the system reports precisely why and who can resolve it.
- Not that the verifier is correct. §16.3 states the verifier integrity boundary explicitly, and §17.2 sketches an approach rather than reporting one.

2. Related work

Privilege enumeration tooling. Established auditors enumerate escalation surfaces far more broadly than the seven surfaces here: LinPEAS sweeps for misconfigurations and ranks candidates by exploitability [peass], Lynis performs host hardening audits [lynis], and OpenSCAP evaluates hosts against declarative policy baselines [oscap]. This system is not competitive with them on enumeration breadth, and §16.1 concedes the gap in their favour. It differs in what it does with silence. Those tools produce a ranked report for a human reader, in which a check that found nothing and a check that could not run are both rendered as the absence of a finding; the reader supplies the judgement. This system produces a machine-consumed verdict, which forces the distinction to be represented: a surface that was not measured and a mechanism whose deciding fact was unreadable each become a *blocking path*, not a quiet omission. OpenSCAP's `notchecked` / `notapplicable` result states are the closest prior mechanism; the difference is that here the unresolved state is load-bearing in the final decision rather than reported alongside it. Breadth and this discipline are complementary, and §17.1's registry is the interface at which a broader enumerator could be attached.

Container privilege boundaries. That access to a rootful container runtime confers host root is established [Bui15, CMD16, NCC16, Lin18] and is not claimed as a finding here; §12.1 re-derives it by measurement because the system may not accept a documented fact it has not observed on the host under test.

Evidence and verification. Certifying algorithms [MMNS11] and proof-carrying code [Nec97] set the standard this system aims at and does not reach — a witness the consumer can check without trusting the producer; §16.3 states the distance. in-toto [TA19] and reproducible-build practice [repro, slsa] supply the binding of evidence to the step that produced it, which §10 applies to measurement steps rather than build

steps. The soundness manifesto [Liv15] is the closest statement of this paper's central discipline in another field: declare what is not covered rather than imply exhaustiveness.

Agent security. Work on indirect prompt injection [GAM+23], injection benchmarks [Deb24], emulated-sandbox risk identification [Rua24] and design-level containment of untrusted data flow [Deb25] addresses whether an agent can be induced to attempt a harmful action. This paper addresses the authority available to it once it does. To our knowledge the host-privilege question is not currently posed as a verification problem in that literature.

Sandboxing and confinement. The standard response to an untrusted execution context is to confine it: namespaces to restrict what the process can name [ns7, users7], seccomp filters to restrict the syscalls it can issue [seccomp2], `no_new_privs` to forbid privilege-gaining `execve` [nnp], Landlock to apply unprivileged filesystem restrictions [landlock], a user-space kernel such as gVisor to interpose on the syscall interface [gvisor], and hardware virtualisation to move the boundary below the kernel [firecracker]. Privilege separation [PFH03] and least privilege [SS75] are the design principles these implement.

Confinement is complementary to this work and does not subsume it, for two reasons that the evaluation makes concrete. First, §12.1 measures an authority path across which **no kernel-enforced boundary is crossed**: the DAC check on the runtime socket *passes*, and authority is exercised by a daemon already running as uid 0. There is no interposition point for a seccomp filter, a capability drop, or a namespace to occupy, because nothing is being bypassed. Second, and more sharply, §7.2 shows that confinement applied to the *measuring* process actively damages the measurement: the same collector run under a restricted context reports zero root-equivalent paths on an unchanged host. Confinement and measurement therefore have to be reasoned about separately — which is the observation that produced Finding 3.

3. Problem statement

Let P be a governed principal that alone may mutate a canonical state store S . Let A be an autonomous agent runtime that must be able to *request* mutations of S through a broker, and must not be able to *perform* them.

The governance claim under test is:

Authority exclusivity. No execution path available to A results in mutation of S other than through the broker acting as P .

The naive verification strategy enumerates known escalation mechanisms, tests each, and reports success when all tests deny. It has three defects.

D1 — Silence is read as absence. A mechanism the enumerator does not know about produces no finding, which is indistinguishable from a finding of "denied": both arrive at the verdict function as an empty list.

D2 — Context is unbound. A measurement taken under a credential or namespace that differs from the agent's real runtime describes that context, not the host. An inventory collected inside a restricted sandbox can report a perfectly clean host that has not changed at all.

D3 — Evidence is under-specified. Reproducibility is commonly demonstrated by a stable digest over repeated runs. If the digest does not cover the measurement inventory, stability demonstrates determinism of the *output* and says nothing about the *inputs* having been the same, or complete.

All three defects were present in this system, were measured, and are treated in §7–§9. D1 and D3 recurred *after* the initial hardening and are reported as Findings 1 and 2.

4. Threat model

The adversary is the agent runtime itself, or any code the agent can be induced to execute — including instructions injected through retrieved content [GAM+23], a compromised dependency executing under the agent's uid, and the agent acting on an incorrect belief. The adversary is inside the user account's trust domain, because it executes as that account.

Table 1: Threat model assumptions.

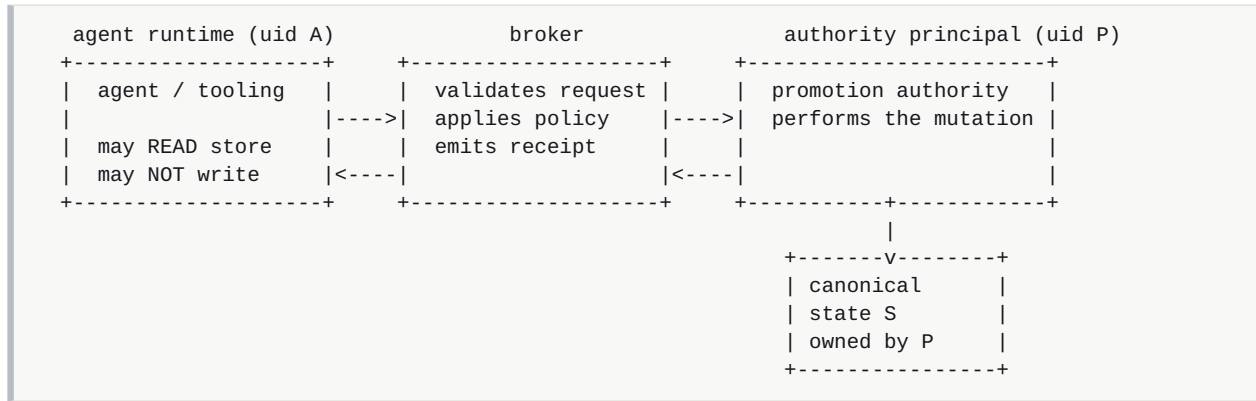
#	Assumption	Rationale
T1	Adversary executes with the agent's real uid and full supplementary group set	This is how agent runtimes are deployed; restricting it would model a different system
T2	Adversary may invoke any binary the DAC permits it to execute	No syscall filtering is assumed on the agent
T3	Adversary may choose flags to any program it invokes	Consequence: a protection removable by a flag is not a boundary
T4	Adversary cannot authenticate as another principal	It holds no password, key or token for <i>P</i> or for root
T5	Adversary cannot modify kernel-enforced DAC/LSM decisions except via authority it already holds	The kernel is trusted
T6	Adversary may read any file the DAC permits, including this verifier's source and its evidence	Evidence integrity is therefore about detection, not concealment
T7	The host administrator is not adversarial but is <i>absent</i> : no privileged read is available at verification time	This is what forces the RE-QUIRES_OPERATOR_EVIDENCE class

Out of scope. Kernel vulnerabilities, hypervisor escape, physical access, supply-chain compromise of the kernel or the C library, and an adversarial host administrator. Cryptographic attacks on SHA-256 are out of scope; evidence integrity here is *tamper-evidence* under T6, not tamper-proofing.

Explicit non-boundary. Under T3, a mitigation the adversary can disable by passing a flag provides no separation. On the evaluated host SELinux denies the container bind-mount write by default and `--security-opt label=disable` removes that denial; because the adversary chooses the flags, this is recorded as evidence and not credited as a boundary.

5. System architecture

The verified system separates request from authority. The broker is the only component holding *P*, and it mediates every mutation — complete mediation [SS75] in the reference-monitor arrangement [And72], applied to agent tooling, with the same privilege-separation structure used to confine privileged network daemons [PFH03].

Figure 1: Authority separation architecture.

Separation rests on discretionary access control: *S* is owned by *P* and is not writable by *A*. The governance claim therefore reduces to a question about *A*'s ability to become *P*, or to become any principal that can write *S* — that is, a question about the host's privilege topology, not about the broker's code. This paper verifies the former and explicitly not the latter (§16.4).

Two inventories exist deliberately. `root_equivalence.py` probes container runtimes by *executing* one; `privilege_topology.py` enumerates the surface read-only. Neither is a superset of the other, so the model unions them: a path found by either is a path.

6. Authority surface model

6.1 Definitions

Authority Carrier. Any mechanism capable of granting execution authority beyond the intended principal boundary. A carrier is identified by the mechanism, not by an exploit. A carrier that requires no exploitation at all — because the authority was granted deliberately — is still a carrier, and is the most severe kind, since no boundary is available to harden.

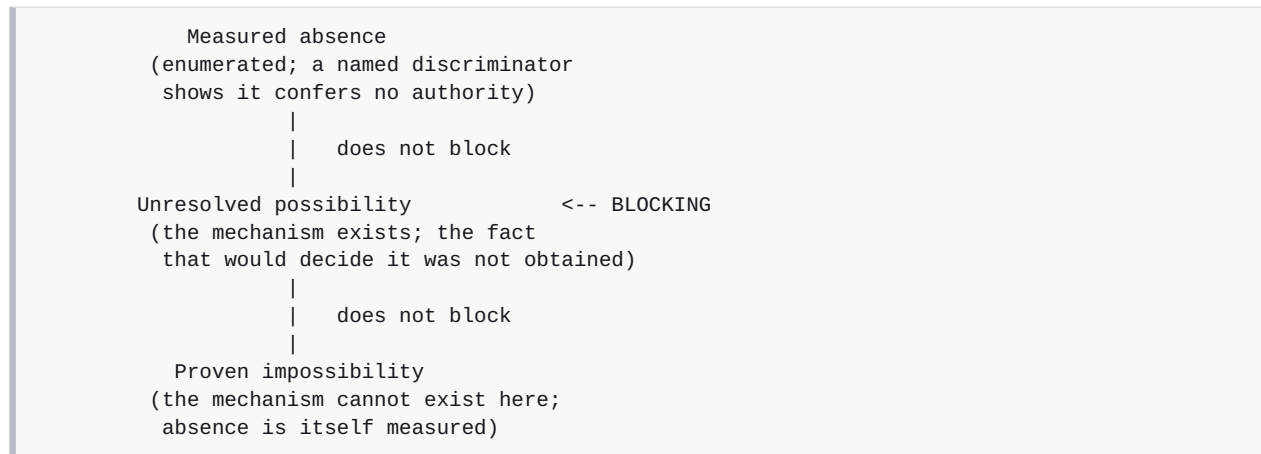
Measurement Completeness. The property that every known authority-producing surface is either measured or explicitly classified as unresolved. Completeness is relative to the surface list; it is not a claim of exhaustiveness over the operating system, and §16.1 states the consequence.

Evidence Integrity. The property that verification evidence changes whenever relevant measurement state changes. A digest that is stable across a change in *what was measured* does not have this property, however reproducible it is. Determinism is necessary for evidence integrity and is not sufficient for it — the distinction that reproducible-build practice draws between a bit-identical output and an attested input set [repro, salsa].

6.2 The three-valued distinction

Conventional verification is two-valued: a mechanism is either found-and-denied or not found. This system distinguishes three states, and the verdict function consumes all three.

Figure 2: Three-valued authority model.

**Table 2:** The three states and their verdict effect.

State	Meaning	Verdict effect
A. Measured absence	The mechanism was enumerated and a named discriminator establishes that it confers no authority on this principal	Does not block
B. Unresolved possibility	The mechanism exists and the fact that would decide it was not obtained	Blocks , always
C. Proven impossibility	The mechanism cannot exist on this host — e.g. the binary is absent <i>and</i> its parent directory is searchable	Does not block

Why state B must block unconditionally. The natural objection is that blocking on every unresolved path is too strong: most unresolved mechanisms will turn out to be bounded, so a risk-weighted treatment — block on the plausible ones, proceed past the rest — would terminate more often. The objection fails for three reasons, and the third is decisive.

1. *The weight would be assigned without the deciding fact.* A path is in state B precisely because the fact that would classify it was not obtained. Any score attached to it is a prior over an unmeasured quantity, and the verifier has no evidence with which to form that prior. Scoring unresolved paths converts a measurement system into an estimation system while leaving the output labelled as verification.
2. *The error is asymmetric.* Blocking a host that is in fact clean costs a privileged read by an administrator. Certifying a host that is in fact reachable costs the property the system exists to guarantee. There is no symmetric threshold to tune between two outcomes of that shape, which is fail-safe defaults [SS75] applied to the verification decision.
3. *A threshold is a discovery target.* Under threat-model assumption T3 the adversary chooses conditions. If some unresolved paths are tolerated, the cheapest attack is no longer to defeat a boundary but to arrange that the path which matters lands in the tolerated class — and, by construction, the verifier cannot tell which one that is. An unconditional block is the only rule with no such target.

The cost is stated rather than hidden: on this host the rule is what makes the verdict terminal at *blocked pending a human* (§16.2), and 15 of the 18 unresolved paths cannot be closed by the agent at all.

The engineering content of the system is almost entirely in keeping B from decaying into A. Three mechanisms enforce this: the verdict function consumes the union of unresolved classes (§8.2), the coverage gate rejects an inventory silent about a required surface (§7.3), and the context gate rejects an inventory that cannot prove where it was measured (§7.2).

A fourth state, **UNDETERMINED existence**, arose from measurement rather than design. `os.path.exists()` returns `False` both for a file that is absent and for a file whose parent directory is unsearchable. On the evaluated host `/etc/libvirt` is mode `0700`, so `/etc/libvirt/libvirtd.conf` had been recorded as `exists: false` — reading as state C when it was state B. Collectors now report PRESENT / ABSENT / UNDETERMINED with `parent_searchable` and a reason.

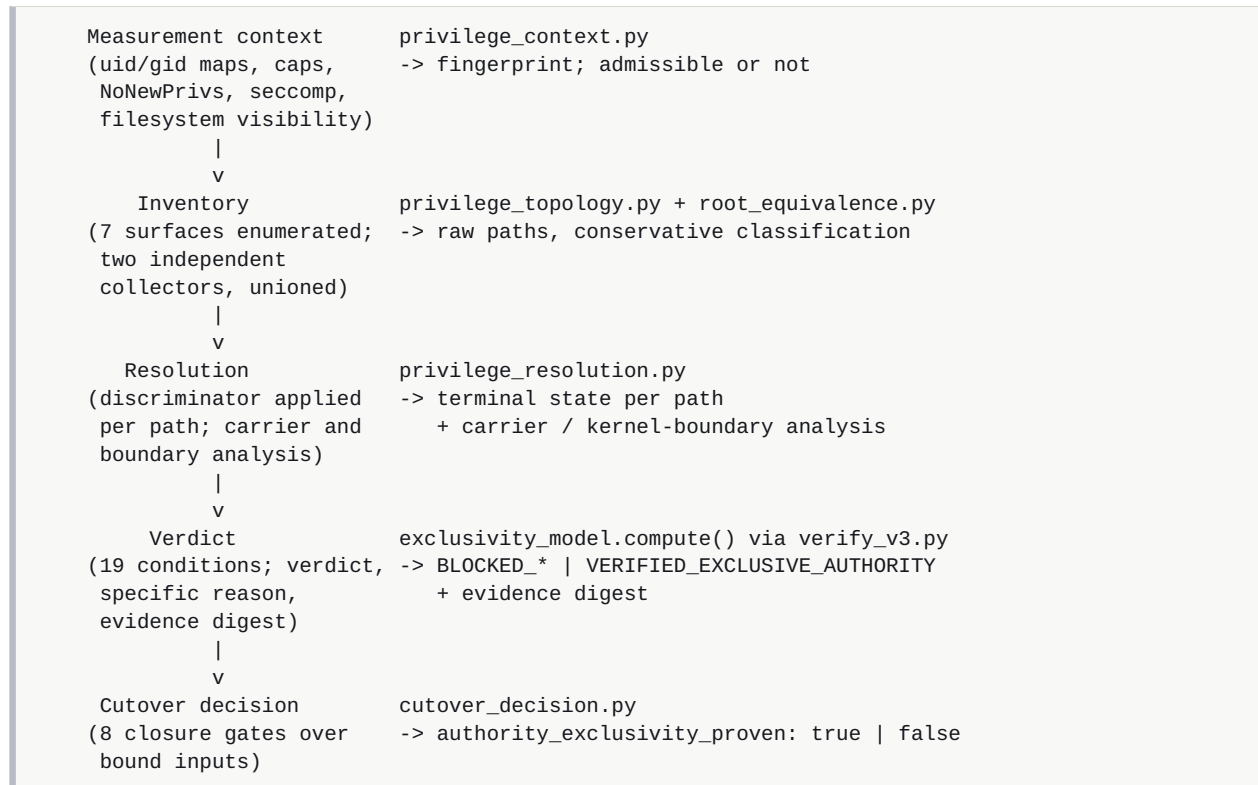
6.3 Surfaces

Table 3: Enumerated surfaces and their discovery methods.

Surface	Enumeration method	Rationale
groups	<code>os.getgroups()</code> against <code>/etc/group</code> rosters	The kernel credential [cred7], not the <code>id</code> utility: the two can disagree under a namespace
contain-er_runtimes	<code>socket stat + connect(2)</code> without sending a byte	Distinguishes DAC-authorised daemons from authorisation-layered ones (§7.4)
sudo	<code>sudo -n -l</code> only	<code>-n</code> never prompts; a refusal is a refusal, not an escalation [sudoers5]
polkit	<code>pkaction --verbose</code> ; world-readable rules under <code>/usr/share</code>	The deciding rules under <code>/etc</code> are unreadable; recorded as such [polkit]
systemd	unit enumeration; <code>ExecStart</code> target writability	A root unit with an agent-writable target is direct root
setuid	<code>find -xdev -type f -perm -4000</code>	Classical; its semantics are famously subtle [CWD02]
filecaps	<code>getcap -r</code> over the same roots	Added after Finding 1. Capabilities live in an extended attribute [cap7, libcap]; the <code>setuid</code> sweep cannot see them

7. Measurement methodology

Figure 3: Evidence integrity pipeline.



Every stage fails closed: a missing input, an unparseable context, or an uncomputed graph closure yields a blocking verdict rather than an absent one.

7.1 Read-only discipline

No collector mutates host state. Sockets are connected and closed without sending a byte. `sudo` is invoked only with `-n -l`, which cannot prompt and cannot escalate. `pkexec` is never invoked. The rationale is not politeness: exercising an escalation path to determine whether it works *performs* the escalation the system is attempting to prove absent, and makes the audit indistinguishable from the attack. Every artifact records `mutations_performed: []` and `read_only: true`.

The cost of this discipline is exactly the `REQUIRES_OPERATOR_EVIDENCE` class: mechanisms that could be resolved by exercising them, and are therefore left unresolved and blocking.

7.2 Measurement context binding

An inventory is evidence about a host only if it was collected as the principal whose authority is in question. This was measured, not hypothesised. The same collector, on the same host, within the same minute:

Table 4: Identical collector, two process contexts, unchanged host.

Observation	Restricted context	Host-representative context
<code>getgroups()</code>	65534 65534 1000	10 969 1000 (wheel, docker, user)
<code>/proc/self/uid_map</code>	1000 1000 1	0 0 4294967295
<code>NoNewPrivs / Seccomp</code>	1 / 2	0 / 0
<code>/var/run/docker.sock</code>	nobody nobody (unmapped)	0:969, srw-rw----
<code>sudo -n -l</code>	"the no new privileges flag is set"	"a password is required"
<code>getent group docker</code>	<code>docker:x:969:user</code>	<code>docker:x:969:user</code>
reported root-equivalent paths	0	2

The last row is the finding; the row above it is the control. The group roster is identical in both contexts, so the host did not change — the credential under which the measurement was taken did. A namespace slice drops supplementary gids [users7, cred7], and `NoNewPrivs = 1` causes every `setuid` transition to be refused by the kernel regardless of the host's configuration [nnp].

An inventory is therefore admissible only if its recorded context satisfies

```
uid_map == gid_map == [(0, 0, 4294967295)] ^ NoNewPrivs == 0 ^ Seccomp == 0
```

Each conjunct can independently suppress a privilege path. The context record also carries capability sets, namespace inodes, and the visibility state of twelve policy surfaces, all folded into one fingerprint: a namespace that hides a policy file must not fingerprint identically to one that can read it.

Inadmissible inventories are not discarded. Discarding would delete their root-equivalent findings and *improve* the verdict. Their paths are ignored and a blocking marker is substituted, so the verdict moves toward uncertainty and never toward closure. §9.4 proves this property.

7.3 Coverage: derived, not declared

Admissibility answers *measured as whom*. Coverage answers *measured of what*. An inventory collected in a perfect context that enumerated only `groups` is silent about `setuid`, and silence is state B, not state A.

Coverage is computed from the path identifiers the inventory actually contains:

```
surfaces_covered := { id.split(":")[0] : id ∈ inventory.paths }
missing          := REQUIRED_SURFACES \ surfaces_covered
∀ s ∈ missing : add blocking marker coverage:<s>_not_measured
```

Deriving coverage from contents rather than from a self-declared field is load-bearing: an inventory that measured nothing would otherwise be free to assert that it measured everything. A regression test asserts that an inventory carrying `surfaces_measured: [all]` and zero paths is treated as covering nothing.

7.4 Discriminators

Every classification is produced by a *discriminator*: a specific fact, read from a specific place, that decides the question. Where no discriminator is readable the path is classified `REQUIRES_OPERATOR_EVIDENCE` together with the exact command that would answer it.

Table 5: Representative discriminators.

Mechanism	Discriminator	Outcome on this host
at, crontab	The daemon executes the job as the <i>submitting</i> user	Bounded: confers no authority the caller lacks
mount.nfs, fuser-mount-glusterfs	/etc/fstab contains no user / users entry	Helper refuses unprivileged callers
qemu-bridge-helper	/etc/qemu/bridge.conf = allow virbr0	Bounded to a named bridge; allow all would have classified root-equivalent
dbus-daemon-launch-helper	Are the D-Bus system-service directories agent-writable?	Root-owned, not writable: no caller-chosen binary
Package provenance	On-disk digest equals the digest rpm recorded for that file at build time	24 of the 35 setuid paths match; see the breakdown below
libvirt sockets	Socket is 0666 by design ; authority is decided per-RPC by polkit [polkit]	A completed connect proves reachability and nothing about authority

Package provenance decides less than it appears to, so its 35 setuid paths are reported in full:

Digest vs package	Terminal state	Count
MATCH	NON_ROOT_EQUIVALENT (bound named, bytes attested)	18
MATCH	REQUIRES_OPERATOR_EVIDENCE (bytes attested, bound not readable)	6
no match	NON_ROOT_EQUIVALENT (bounded by a different discriminator)	6
no match	REQUIRES_OPERATOR_EVIDENCE (binary unreadable by the agent)	5
	total	35

Two classes of measurement are deliberately refused as insufficient. Socket *reachability* is never used as a classifier for authorisation-layered daemons (libvirt, D-Bus), where a completed `connect(2)` is expected and uninformative. And a *digest match attests bytes, not privileges*: one binary on this host matches its package byte-for-byte while the package declares mode `100755` and the disk carries `4755`, so the setuid bit has no vendor provenance and the binary is unresolved rather than bounded.

8. Classification model

8.1 Terminal states

Every path reaches exactly one of four terminal states.

Table 6: Terminal classifications.

Classification	Definition	Blocks
ROOT_EQUIVALENT	Measured to confer uid-0-equivalent authority	Yes
NON_ROOT_EQUIVALENT	Measured bounded, with the bound named and the provenance of the bytes verified	No
NOT_PRESENT	The mechanism does not exist on this host, and its absence is itself measured	No
REQUIRES_OPERATOR_EVIDENCE	The deciding fact is unreadable by the agent; the command that answers it is recorded	Yes

`REQUIRES_OPERATOR_EVIDENCE` is a rename of the older `UNKNOWN`, not a resolution of it. The available failure mode was to rename every unresolved path into the new class, report "0 UNKNOWN", and let the verdict proceed. Three mechanisms prevent this: the collector returns `unresolved := unknown U requires_operator_evidence` and the verdict branches on the union; condition 19 consumes the union; and a regression test asserts that both spellings of the same path produce an identical verdict.

The default for an unrecognised mechanism is `REQUIRES_OPERATOR_EVIDENCE`. Adding a new `setuid` binary or file capability to the host therefore degrades the verdict rather than being silently tolerated.

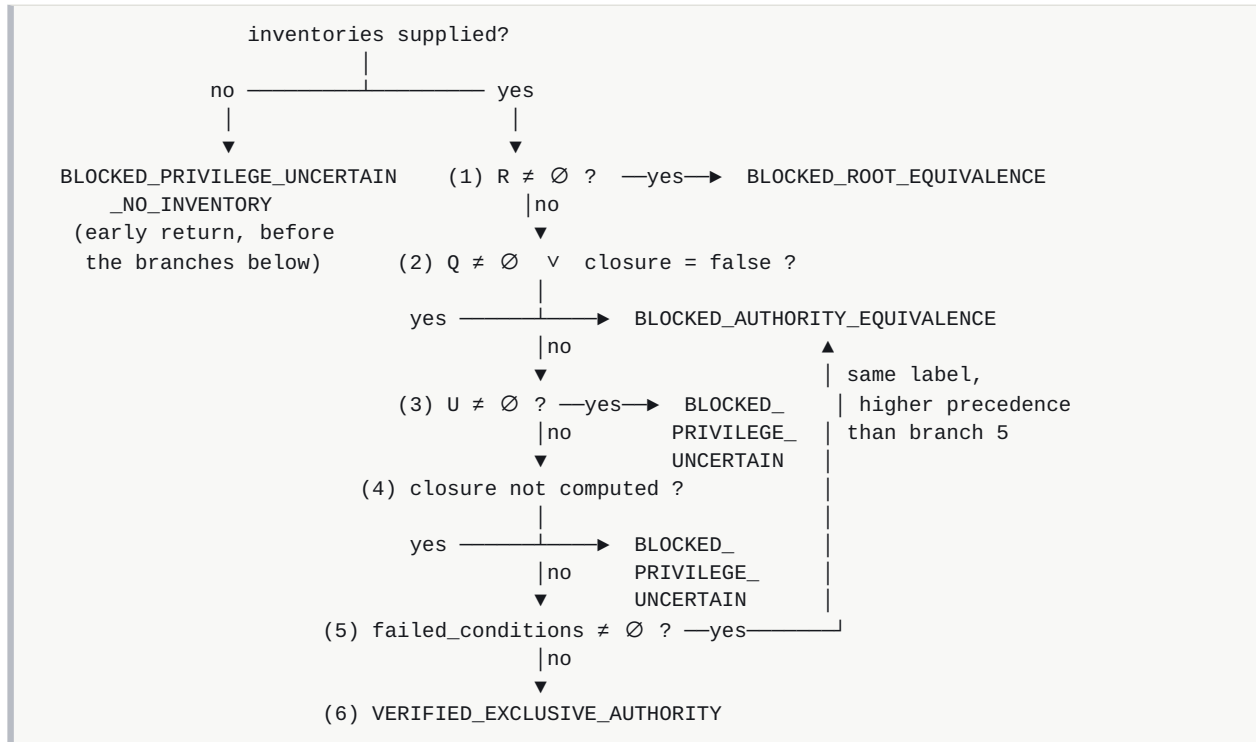
8.2 Verdict function

```

if root_equivalent_paths ≠ ∅:                BLOCKED_ROOT_EQUIVALENCE
elif authority_equivalent ≠ ∅ ∨ graph_closed = false: BLOCKED_AUTHORITY_EQUIVALENCE
elif unresolved_privilege_paths ≠ ∅:          BLOCKED_PRIVILEGE_UNCERTAIN
elif graph_closure not computed:              BLOCKED_PRIVILEGE_UNCERTAIN
elif failed_conditions ≠ ∅:                   BLOCKED_AUTHORITY_EQUIVALENCE
else:                                         VERIFIED_EXCLUSIVE_AUTHORITY

```

Figure 4: Verdict decision flow. Branch order is the order in `compute()`; the labels on the right are the emitted verdicts. Branches 2 and 5 emit the *same* label at different precedence positions, which is the subject of Theorem 2 (§9.5).



U carries the marker injections of §9.3, so an inadmissible inventory and an unmeasured required surface both reach branch 3 rather than bypassing the decision. Branch 6 is the only accepting state, and it is reachable only when every prior test is false.

Precedence is deliberate. A measured escalation outranks an unmeasured one: reporting "uncertain" on a host where the agent demonstrably holds root would understate the finding. Uncertainty outranks a merely failing condition, because a condition that fails under an unresolved privilege path has not been measured on a known host. §9.5 shows the price of that choice.

A `specific_reason` — `BLOCKED_ROOT_EQUIVALENCE_DOCKER` on this host — is *derived* from the path set rather than written by hand, on the principle that a verdict a human can spell is a verdict a human can spell wrongly. Every missing input fails closed: absent inventories, uncomputed graph closure, and unparsable context all yield `BLOCKED_PRIVILEGE_UNCERTAIN`. This is fail-safe defaults [SS75] applied to the verification decision rather than to the access decision — the base case is refusal, and evidence is what moves away from it.

9. Formal authority exclusivity model

This section states the model the implementation computes, and proves the one safety property that actually holds. §9.6 records where the formalisation and the code differ.

9.1 Objects

Sym- bol	Object	Definition
A	Agent	The runtime under governance, executing with a fixed credential (uid, gid set, capability sets)
P	Authority principal	The only principal permitted to mutate S

Sym-bol	Object	Definition
S	Governed state	The canonical store; <code>write(S)</code> is permitted to P alone by DAC
C	Authority carrier	A mechanism by which a principal may obtain execution authority beyond its own boundary
H	Host	The machine under measurement, at one instant
X	Measurement context	The process context in which a measurement is taken: uid/gid maps, supplementary gids, capability sets, <code>NoNewPrivs</code> , <code>seccomp</code> mode, namespace identities, and the visibility state of the policy surfaces

Let $\mathcal{C}(H)$ be the set of authority carriers actually present on H . $\mathcal{C}(H)$ is not observable; it is the thing the system is trying to bound.

9.2 Measurement and classification

The measurement function is

$$M : (H, X) \rightarrow E$$

where E is an **inventory**: a finite set of *paths*, each a pair (path identifier, evidence record), together with the record of X under which it was taken. A path identifier has the form `surface:instance`, so the surface of a path is recoverable from the path itself — the property §7.3 relies on.

The classification function assigns each path exactly one terminal state:

$$\text{cls} : \text{path} \rightarrow \{ \text{ROOT_EQUIVALENT}, \\ \text{NON_ROOT_EQUIVALENT}, \\ \text{NOT_PRESENT}, \\ \text{REQUIRES_OPERATOR_EVIDENCE} \}$$

`cls` is total by construction: an unrecognised mechanism receives `REQUIRES_OPERATOR_EVIDENCE`, so no path can escape classification by being unfamiliar.

Three predicates are defined over an inventory E :

$$\begin{aligned} \text{admissible}(E) &\equiv \text{uid_map}(X) = \text{gid_map}(X) = [(\emptyset, \emptyset, 2^{3^2}-1)] \\ &\quad \wedge \text{NoNewPrivs}(X) = \emptyset \wedge \text{Seccomp}(X) = \emptyset \\ \text{covered}(E) &\equiv \{ \text{surface}(p) : p \in \text{paths}(E) \} \\ \text{complete}(E) &\equiv \text{REQUIRED_SURFACES} \subseteq \text{covered}(E) \end{aligned}$$

`admissible` is the formal content of " E is evidence about H rather than about X ". Note that both `covered` and `complete` are computed *from* `paths(E)`; neither reads a field in which E declares its own coverage.

9.3 The verdict predicate

From a set of inventories $\mathcal{E}$ the model derives four path sets and two booleans:

```

R = U { root(E)      : E ∈ ℰ, admissible(E) }
Q = U { authority(E) : E ∈ ℰ, admissible(E) }
U = U { unresolved(E): E ∈ ℰ, admissible(E) }
    U { marker(E)    : E ∈ ℰ, ¬admissible(E) }           (i)
    U { marker(s)    : s ∈ REQUIRED_SURFACES \ U covered(E) } (ii)

ctx_valid    ≡ ℰ ≠ ∅ ∧ ∀E ∈ ℰ : admissible(E)
graph_closed ≡ the privilege graph closure was computed and holds

```

Here `root(E)`, `authority(E)` and `unresolved(E)` are the path sets the inventory *publishes*, with `unresolved(E) = unknown(E) ∪ requires_operator_evidence(E)`. They are written as inventory-supplied sets rather than as `{p ∈ paths(E) : cls(p) = ...}` because that is what the model reads; §9.6 records the consequence. `covered(E)`, by contrast, is computed from `paths(E)` and is never published by the inventory (§7.3).

Clauses (i) and (ii) are the load-bearing detail. An inadmissible inventory does not vanish: it contributes a blocking marker to `U`. A required surface that no admissible inventory covers contributes a blocking marker to `U`. Both forms of *missing evidence* are represented as *present unresolved paths*.

Note that `ℰ` is heterogeneous. The registry inventory publishes `root(E)` but no `paths(E)` at all, so it contributes to `R` while contributing nothing to coverage. This asymmetry is deliberate — it is the two-inventory union of §5 — and it has a measurable consequence in §9.5.

The positive verdict is then

```

VERIFIED_EXCLUSIVE_AUTHORITY ⇔ R = ∅ ∧ Q = ∅ ∧ U = ∅
                             ∧ ctx_valid ∧ complete ∧ graph_closed

```

and every other input yields one of `BLOCKED_ROOT_EQUIVALENCE`, `BLOCKED_AUTHORITY_EQUIVALENCE`, or `BLOCKED_PRIVILEGE_UNCERTAIN` by the precedence in §8.2. Write **V** for `VERIFIED_EXCLUSIVE_AUTHORITY` and **B** for the set of the three blocked labels.

Lemma 1 (missing evidence is present evidence). $\neg \text{ctx_valid} \Rightarrow U \neq \emptyset$, and $\neg \text{complete} \Rightarrow U \neq \emptyset$.

Proof. If $\neg \text{ctx_valid}$ then either $\mathcal{E} = \emptyset$, in which case the model returns `BLOCKED_PRIVILEGE_UNCERTAIN_NO_INVENTORY` without evaluating the predicate, or some $E \in \mathcal{E}$ has $\neg \text{admissible}(E)$, and clause (i) places `marker(E)` in `U`. If $\neg \text{complete}$ then some $s \in \text{REQUIRED_SURFACES}$ is absent from $\bigcup \text{covered}(E)$, and clause (ii) places `marker(s)` in `U`.

Lemma 1 is why the implementation need not test `ctx_valid` and `complete` separately in the verdict branch: it tests $U = \emptyset$, which by Lemma 1 subsumes both. The predicate in §9.3 lists them for clarity, not because the code evaluates six conjuncts.

9.4 Safety properties

Define four operations on the model's input, each corresponding to something an adversary, a mistake, or an environment change can do:

- **op1 (add unresolved):** add a path p with `cls(p) = REQUIRES_OPERATOR_EVIDENCE`.
- **op2 (remove coverage):** delete paths such that some surface $s \in \text{REQUIRED_SURFACES}$ is no longer covered — the limiting case being an inventory that measured nothing.
- **op3 (add root-equivalent carrier):** add a path p with `cls(p) = ROOT_EQUIVALENT`.
- **op4 (invalidate context):** replace an inventory E by E' with the same paths and $\neg \text{admissible}(E')$.

Theorem 1 (V-unreachability). For any input I and any $op \in \{op1, op2, op3, op4\}$, if $\text{verdict}(op(I)) = V$ then $\text{verdict}(I) = V$ and op was vacuous. Equivalently: none of the four operations can turn a blocked verdict into V , and none can preserve V unless it changed nothing relevant.

Proof sketch. V requires $R = \emptyset \wedge Q = \emptyset \wedge U = \emptyset \wedge \text{graph_closed}$.

- $op1$ adds a member to U , so $U \neq \emptyset$ and the third conjunct fails. The verdict is `BLOCKED_PRIVILEGE_UNCERTAIN` unless a higher-precedence branch already fires.
- $op2$ removes surface s from $\bigcup \text{covered}(E)$, so by clause (ii) $\text{marker}(s) \in U$, and the third conjunct fails. The limiting case — an empty inventory — adds one marker per required surface, so an inventory that measured nothing is maximally blocking rather than maximally clean.
- $op3$ adds a member to R , so the first conjunct fails and the first precedence branch fires: `BLOCKED_ROOT_EQUIVALENCE`.
- $op4$ makes $\text{admissible}(E')$ false, so by clause (i) $\text{marker}(E') \in U$ and the third conjunct fails. Note that E 's paths are also excluded from R , Q and U , which is what makes $op4$ interesting; see Theorem 2.

In each case a conjunct required by V is falsified by construction, so V is not reachable. If op is vacuous — $op1$ adding a path already in U , $op2$ deleting a path whose surface remains covered, $op3$ adding a path already in R , $op4$ applied to an inventory already inadmissible — the input is unchanged and the verdict is unchanged. .

Corollary 1 (no certification from incomplete evidence). A verifier implementing this model cannot emit V while any of the following holds: an unresolved path exists; a required surface is unmeasured; an inventory's measurement context is invalid or unparseable; no inventory was supplied; or the privilege graph closure was not computed. *Proof:* the first three by Theorem 1 and Lemma 1; the fourth by the early return in `compute()`; the fifth by the fourth precedence branch, which maps an uncomputed closure to `BLOCKED_PRIVILEGE_UNCERTAIN` rather than treating `None` as `true`..

Corollary 1 is the property the system exists to have, and it is strictly weaker than "the host is safe". It says only that *this* verifier will not convert an incomplete measurement into a certification.

9.5 What is *not* true: the labels are not a total order

The implementation exposes an ordered tuple `PRECEDENCE = (BLOCKED_ROOT_EQUIVALENCE, BLOCKED_AUTHORITY_EQUIVALENCE, BLOCKED_PRIVILEGE_UNCERTAIN, VERIFIED_EXCLUSIVE_AUTHORITY)`. It is tempting to read this as a severity lattice and to claim the stronger property that no operation can move the verdict toward V *at all*. That claim is false, and the counterexamples are reachable rather than pathological.

Theorem 2 (non-monotonicity in the label order). There exist inputs I and operations $op \in \{op1, op2\}$ such that $\text{verdict}(op(I))$ is strictly closer to V in `PRECEDENCE` than $\text{verdict}(I)$.

Proof by counterexample.

- **C1 (op1 on a failing-condition input).** Take $R = Q = U = \emptyset$, `graph_closed = true`, `failed_conditions` $\neq \emptyset$. The fifth branch fires: `BLOCKED_AUTHORITY_EQUIVALENCE`. Apply $op1$. Now $U \neq \emptyset$, the third branch fires first, and the verdict is `BLOCKED_PRIVILEGE_UNCERTAIN` — one position closer to V .
- **C2 (op2 on the evaluated host).** Computed by running the pure model over the shipped artifacts with paths deleted; the intermediate step is reported because it is where the naive version of this counterexample fails.

Input	Verdict / specific reason	R
as measured	BLOCKED_ROOT_EQUIVALENCE / ... _DOCKER	contain- er_runtimes:docker_rootful_socket , gr oups:membership_docker , dock- er_rootful
delete container_runtimes from the topology and clear the registry's root list	BLOCKED_ROOT_EQUIVALENCE / ... _DOCKER	groups:membership_docker
additionally delete groups	BLOCKED_PRIVILEGE_UNCERTAIN /_INCOM- PLETE_SURFACE_COVERAGE	∅

The third row is the counterexample: R is empty, clause (ii) has placed `coverage:container_runtimes_not_measured` and `coverage:groups_not_measured` in U , and the verdict has moved one position closer to V .

The cause is that `BLOCKED_AUTHORITY_EQUIVALENCE` is *overloaded*: it labels both a structural authority-equivalence or open graph (branch 2, above uncertainty) and a mere condition failure (branch 5, below uncertainty). One label therefore occupies two precedence positions, and no total order over four labels can be consistent with that.

The consequence is operational, not merely formal. C2 says that an adversary who can *delete evidence* — truncate an inventory, suppress a collector — cannot manufacture a certification, but *can* launder a specific, actionable blocker (`..._DOCKER`, which names the mechanism and the remediation) into a generic one (`..._INCOMPLETE_SURFACE_COVERAGE`, which names only that something is missing). The safety property survives; the *diagnostic* value does not.

The intermediate row bounds how cheap that attack is, and the bound is a measured property of the design rather than a hope. The grant is visible at two layers and in two inventories, so a single deletion is insufficient three times over: the socket path and the group path are separate members of R , and the registry's `docker_rootful` is a third, contributed by an inventory that publishes no `paths` dict and therefore cannot be suppressed by deleting surfaces at all. This is the two-inventory union of §5 and the one-mechanism-two-carriers listing of §12.1 doing exactly the work they were introduced for. §14.3 makes the attack an explicit target of the proposed red-team protocol (goal G2), and §17.1 proposes the registry field that would let a surface's disappearance be distinguished from a surface that never existed.

9.6 Fidelity of the formalisation

Four differences between §9.3 and `exclusivity_model.py` are recorded rather than elided.

1. The code does not evaluate `ctx_valid` and `complete` as verdict conjuncts; it evaluates $U = \emptyset$. Lemma 1 is the justification, and the equivalence is pinned by regression tests (`test_contextless_inventory_is_inadmissible`, `test_partial_inventory_cannot_improve_a_verdict`).
2. `every_inventory_host_measured` is reported as a separate boolean for the cutover gate, but the verdict does not read it. It is a report of Lemma 1's antecedent, not an input to the decision.
3. Q is specified and unexercised. Of the shipped artifacts, exactly one (`ROOT_EQUIVALENCE_REGISTRY.json`) carries an `authority_equivalent_paths` field, and its value is `[]`; no discriminator in the current classifier emits the class. Branch 2 of the verdict function therefore fires only via `graph_closed = false` on every evaluated run. The set is retained because the privilege graph can populate it, but nothing in §13 exercises it.

4. *R*, *Q* and *U* are **declared by the inventory**, not derived from per-path classifications: the model reads `root_equivalent_paths`, `authority_equivalent_paths`, `unknown_paths` and `requires_operator_evidence_paths` as published lists. Coverage is derived (§7.3) and these are not. On the evaluated host they agree — the topology's published `root_equivalent_paths` is exactly the set of its paths classified `ROOT_EQUIVALENT` — but that agreement is a property of the producer, not an invariant the consumer enforces. By the paper's own recorded-versus-consumed rule (§12.5) this is the next thing that should be derived, and it is not yet. It is disclosed here rather than in future work because it is a present gap, not a planned feature.

10. Evidence integrity model

A recorded fact is insufficient if a different fact is consumed by the verifier.

This is the paper's central claim about verifier construction, and it is not a statement about carelessness. In each of the three instances below, the correct fact was measured, written to an artifact, and available; the defect was that the decision consumed something adjacent to it. No amount of care applied to the *measurement* would have prevented any of them, because in each case the measurement was right.

Failure mode	Recorded	Consumed	Detected by
Inventory completeness (Finding 1)	6 surfaces	6 surfaces, of the 7 that exist	Coverage gate, after the surface was added
Digest completeness (Finding 2)	a 57-path inventory	attack verdicts, registry classifications, condition booleans	Repeatability condition, when it failed to fail
Context binding (Finding 3)	inventory <i>and</i> its measurement context	the inventory, unbound to the context	Control measurement of an unchanged host

The three are not variations of one bug; they sit at three different layers, and each was invisible from the others. The generalisation and the recurring remedy are in §12.5. The layers below are what makes the class detectable at all.

Evidence is bound in three layers. The design goal is the one certifying algorithms formalise [MMNS11] and proof-carrying code applies to mobile code [Nec97]: the consumer should be able to check the result without trusting the producer. This system reaches only part of that goal — it emits checkable evidence, not a checkable proof — and §16.3 states the gap.

Layer 1 — artifact digests. Each artifact records the SHA-256 of every input it consumed, with size and mtime. A verdict is closure only against the inputs it was computed from. This is the same binding in-toto applies to supply-chain steps [TA19], applied to measurement steps.

Layer 2 — append-only manifests. Manifests are never regenerated, because rewriting an earlier manifest destroys the link between a digest and the run that produced it. Supersessions are enumerated instead; §13.3 reports the current supersession state.

Layer 3 — the evidence digest. A SHA-256 over an explicit whitelist of the security-relevant surface. A whitelist rather than a blacklist: filtering volatile keys out of raw JSON drifted on every run (temporary paths in error strings, pids in diagnostics), which made the repeatability condition unsatisfiable for reasons unrelated to the security result.

Layer 3 is where Finding 2 occurred; the corrected definition is in §12.3. The property the layer must have is Evidence Integrity as defined in §6.1 — evidence changes whenever relevant measurement state changes. The system's repeatability condition (condition 18) requires four consecutive runs producing an identical verdict and evidence digest. Because the digest now covers the inventory, any change in what was measured resets that count; that is the intended behaviour and was observed (§13.3).

11. Implementation

Python 3, no third-party runtime dependencies; `ruff` for lint, `unittest` for tests. Approximately 6,800 lines across collectors, model, verifier, gates and tests, of which about 1,030 are tests.

Figure 5: Component graph.

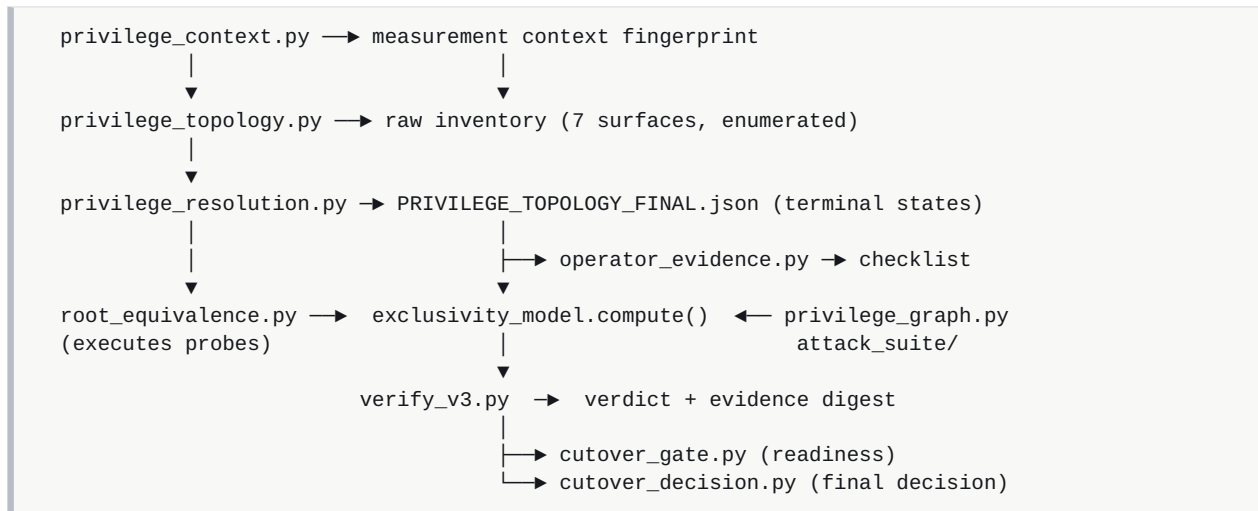


Table 7: Components and purity.

Component	Responsibility	Purity
<code>privilege_context.py</code>	Context fingerprint, filesystem visibility, admissibility predicate	Reads <code>/proc/self</code> only
<code>privilege_topology.py</code>	Enumerates 7 surfaces; classifies conservatively	Read-only, no escalation
<code>privilege_resolution.py</code>	Applies discriminators; emits terminal states and carrier analysis	Read-only; <code>rpm / getcap</code> queries
<code>exclusivity_model.py</code>	Collection, coverage, verdict, specific reason	Pure: no I/O, no clock, no host access
<code>operator_evidence.py</code>	Deterministic checklist; readable vs privileged-only vs inference	Read-only
<code>verify_v3.py</code>	Stage orchestration, 19 conditions, evidence digest	Executes probes
<code>cutover_gate.py</code>	Readiness gate; verdict without starting a container	Read-only
<code>cutover_decision.py</code>	Final decision from bound inputs	Read-only

The purity of `exclusivity_model.py` is what makes the regression suite meaningful: every verdict case is a dict-in/dict-out test requiring no host, no container and no privilege. Host-dependent behaviour is tested separately against recorded artifacts.

Anti-self-attestation. Three properties are derived rather than declared: surface coverage (from path identifiers); the privileged-access flag on a checklist item (from the measured integrity state, which corrected a

hand-written table entry claiming a mode-4711 binary needed analysis only); and the specific reason (from the path set).

Dispatcher-level testing. Unit tests on the model prove nothing about the verifier. The suite imports the real `verify_v3`, runs its real condition builder over recorded evidence, and asserts that the verifier source contains the model call, the resolution stage, and the resolved-inventory constant. A handler that exists but is not wired into the path that receives traffic is a distinct failure class from a handler that is wrong, and is checked explicitly.

12. Findings

12.1 The residual carriers

Two paths are measured `ROOT_EQUIVALENT`. They are one mechanism observed at two layers, and both are listed because removing either alone does not necessarily remove the other: an ACL entry can replace group membership as the grant [acl5].

Figure 6: The measured root-equivalent carrier chain.

```

agent identity (uid A)
  | supplementary group membership, gid 969 (docker)
  ▼
DAC check on connect(2) to /var/run/docker.sock [mode 0660, owner 0:969]
  | PASSES – this is a grant, not a bypass
  ▼
Docker Engine API → daemon already running as uid 0
  | create + start with host bind mount
  ▼
host mutation authority (container process is real host uid 0)

```

Property	Value
Carrier	AF_UNIX socket /var/run/docker.sock, mode 0660, 0:969 [unix7]; and gid 969 in the credential
Kernel-enforced boundary crossed	NONE
rootless_daemon	false
container_uid_map	0 0 4294967295 (real host uid 0)
Host file owner after write	4242 (chown target reached)
Attack H_root-ful_container_root	EXPECTED_SUCCESS, changed_canonical: true
Minimal removal	Remove the agent from group docker, or drop the ACL entry; the daemon needs no change

That no boundary is crossed is the substantive result, not a caveat. The kernel's DAC check on `connect(2)` passes, because gid 969 is in the credential and the socket is group-writable. Authority is then exercised by a daemon already running as uid 0. There is consequently no exploit to detect, no boundary to harden, and nothing for a sandbox, capability drop or seccomp filter to interpose. The question is not "can the adversary break isolation" but "was authority already delegated" — and it was.

Docker's own documentation describes the capability without stating the equivalence: "you can start a container where the `/host` directory is the `/` directory on your host; and the container can alter your host filesystem without any restriction" [dockersec]. The equivalence to host root is measured here rather than assumed: the daemon is not rootless, the container's uid map is the host identity map, and a file written through the mount

landed on the host owned by the target uid. The literature on container privilege boundaries reaches the same conclusion by other routes [Bui15, CMD16, NCC16, Lin18]. The remedies are to remove the grant or to run the daemon rootless [rootless]; neither is available to the agent.

12.2 Finding 1 — inventory completeness failure

Class: the enumerator cannot see the surface.

`audit_setuid()` searches `find -xdev -type f -perm -4000`. Linux file capabilities are stored in an extended attribute and carry no setuid bit [cap7, libcap, HM08], so this sweep is *structurally incapable* of finding them. The authority model contained no `filecaps` surface, and the inventory reported full coverage of the surfaces it had chosen to require.

A `getcap -r` sweep found 11 capability-bearing binaries, **none of which appeared anywhere in the inventory**:

```
/usr/bin/suexec          cap_setgid,cap_setuid=ep
/usr/libexec/sss/krb5_child  cap_dac_read_search,cap_setgid,cap_setuid=p
/usr/bin/warp-svc         cap_dac_read_search,cap_setgid,cap_setuid,
                          cap_net_bind_service,cap_net_admin,
                          cap_net_raw,cap_sys_ptrace=ei
/usr/bin/{newuidmap,newgidmap} cap_setuid / cap_setgid=ep
+ 4 network helpers (cap_net_raw, cap_net_bind_service, cap_sys_nice)
```

`cap_setuid` in a file's permitted set is uid transition authority — precisely the property the system exists to bound — reachable with no setuid bit anywhere.

Table 8: Classification of the file-capability surface.

#	Discriminator	Result on this host
1	Can the agent exec the file at all?	<code>suexec</code> is <code>r-x--x---</code> root:apache; the three <code>sss</code> helpers are <code>rw-r-x---</code> root:sss. <code>os.access(X_OK)</code> is false for all four. A capability on a file the caller may not execute is not that caller's path
2	Are the capabilities inheritable-only?	<code>warp-svc</code> is <code>=ei</code> with no <code>p</code> , so exec grants only what the caller already holds [cap7]. The agent's <code>CapInh</code> is <code>0x8000000000</code> ; intersected bit by bit with the file's seven capabilities the result is empty
3	Otherwise permitted-on-exec	The remaining bound would be the program's own logic, which is not readable; such a path would be <code>REQUIRES_OPERATOR_EVIDENCE</code>

All 11 classified `NON_ROOT_EQUIVALENT` **after measurement**. The surface contained no new blocker — but that was not knowable until it was measured, and a host carrying `suexec` at mode `4755` would have produced a different answer that the previous inventory could not have seen.

One near-miss is recorded because it nearly changed a classification: `CapInh 0x8000000000` is bit 35, `cap_wake_alarm`, **not** `cap_bpf` (bit 39). Misreading the bit would have placed a capability in the "agent could activate" set. The three capability numbers that decided something are pinned by test.

Generalisation. The defect is not "we forgot capabilities". It is that `REQUIRED_SURFACES` is a curated list, and a coverage gate can only prove coverage *of the list*. §16.1 and §17.1 address the consequence.

12.3 Finding 2 — evidence reproducibility failure

Class: the integrity mechanism does not cover the thing whose integrity is claimed.

Before the correction, four consecutive runs produced an identical evidence digest and the repeatability condition passed. Adding the entire `filecaps` surface then took the inventory from 46 paths to 57 — and left the digest **bit-identical**.

The cause: `evidence_digest()` hashes an explicit whitelist — attack carrier verdicts, registry classifications, graph edges, condition booleans — and the privilege inventory was not among them. Only condition 19's boolean reached the digest, and it did not flip, because the host had unresolved paths both before and after.

The claim "four consecutive runs produced the same evidence digest" therefore proved determinism of the verdict pipeline and said nothing about whether the same privilege surface had been measured each time. A 46-path run and a 57-path run were indistinguishable to the repeatability condition.

The whitelist now includes the inventory's classification summary: surfaces measured, counts, every path with its terminal state, and whether the measurement was host-representative. Raw evidence blobs (modes, hashes, command output) remain excluded, since those are what made the earlier blacklist approach drift. A regression test pins the property in both directions — moving one path to a different terminal state changes the digest, dropping a surface changes the digest, and identical input still reproduces the digest exactly.

The correction invalidated the four-run baseline, which was re-established from scratch; the observed sequence FAIL, FAIL, FAIL, PASS across the four runs is itself evidence that the digest is now sensitive to the inventory (§13.3).

12.4 Finding 3 — measurement context (prior pass, summarised)

Reported for completeness because Findings 1 and 2 are instances of the same class. An identical collector reported `root_equivalent_paths: []` inside a restricted process context and two carriers outside it, with no host change; the control (`getent group docker`, identical in both) establishes that the host did not change. Remedied by the admissibility predicate of §7.2 and clause (i) of §9.3.

12.5 The common structure

Table 9: Recorded fact versus consumed fact.

Finding	What was recorded	What was consumed	Consequence if unfixed
Original defect	<code>unknown_paths</code>	nothing	An unresolved path certified as clean
Finding 3	inventory, with its context	inventory, unbound to context	A sandbox produces a clean host
Finding 1	6 surfaces	6 surfaces, of the 7 that exist	A whole surface silently absent while coverage reports complete
Finding 2	a 57-path inventory	condition booleans only	An 11-path change invisible to the integrity mechanism

In each case a fact was recorded and a *different* fact was consumed. The recurring remedy is to make the consumed set derived from, and provably equal to, the recorded set — the same move as deriving coverage from path identifiers (§7.3) and deriving the specific reason from the path set (§8.2).

Restated as a design rule: **every field a verifier records must be either consumed by a decision or deleted.** A recorded-but-unconsumed field is not inert documentation; it is a fact the reader will assume was checked.

13. Security evaluation

13.1 Measured authority topology

Table 10: Terminal classifications by surface (57 paths).

Surface	ROOT_EQUIVALENT	NON_ROOT_EQUIVALENT	RE-QUIRES_OPERATOR_EVIDENCE	NOT_PRESENT
container_runtimes	1	1	2	0
filecaps	0	11	0	0
groups	1	0	1	0
polkit	0	0	2	0
setuid	0	24	11	0
sudo	0	0	1	0
systemd	0	1	1	0
Total	2	37	18	0

57 paths; 20 blocking. `unknown_privilege_paths` in the topology is 0. The unioned model carries one further UNKNOWN contributed by the registry (`sudo_polkit_interactive`, the registry's coarser label for the same sudo/polkit question), giving 19 unresolved paths in total.

Of the 18 operator-evidence items, **15 require privileged access** — gated on `/etc/sudoers`, `/etc/sudoers.d`, `/etc/polkit-1/rules.d`, `/etc/gshadow`, `/etc/libvirt/*`, `/etc/shadow`, or the bytes of a `mode-4711` binary — and **3 require analysis only**. All 18 block.

13.2 Verdict

verdict	BLOCKED_ROOT_EQUIVALENCE
specific_reason	BLOCKED_ROOT_EQUIVALENCE_DOCKER
authority_exclusivity_proven	false
evidence_digest	ab4ae6bcb450971214b4e4e6a34be75147653826400ccd09c5e3786415e1c9ac

Table 11: Closure gates.

Closure gate	State
every_inventory_host_measured	PASS
all_required_surfaces_covered	PASS
no_root_equivalent_path	FAIL
no_unknown_path	FAIL
no_operator_evidence_pending	FAIL
privilege_graph_closed	FAIL
readiness_gate_cleared	FAIL
verifier_verdict_is_exclusive	FAIL

Conditions: 14 of 19 pass. Failing: `06`, `07`, `12`, `17`, `19`. Property split: `A_dac_effectiveness` PASS, `B_principal_exclusivity` FAIL (blocked by `docker_rootful`), `C_governed_liveness` PASS.

`failed_for_architectural_reasons` is empty: every failing condition is one that host-level remediation removes. This supports the narrower claim that the *design* separates authority correctly while the *deployment* does not. It is a claim about a conditional, not a certification.

13.3 Validation

Table 12: Validation results.

Check	Result
Lint (<code>ruff check .</code>)	All checks passed
Regression suite	72 tests, OK
Gate control cases	14 of 14, 0 mismatches
Manifest verification	<code>sha256sums.final.txt</code> : 27 of 29 OK; 2 document entries superseded by <code>sha256sums.paper.txt</code> (§10, Layer 2)
Prior package immutability (V1, V2)	0 files changed; verifier's own check <code>prior_packages_unchanged: true</code>
Repeatability (condition 18)	FAIL, FAIL, FAIL, PASS across four runs after the digest correction

The control suite includes a **positive control** (`POSITIVE_fully_cut_over` → `READY_FOR_FINAL_PROOF`). A gate that can only ever emit BLOCKED is not evidence; the positive control is what makes the negative results meaningful.

Negative controls cover: docker socket reachable; docker group present; sudo unresolved; a path visible to only one inventory; root-equivalence outranking uncertainty; missing inventory; operator-evidence pending; inventory measured in a sandbox; a sandbox inventory concealing a root path; partial surface coverage; and a dropped surface.

Table 13: Selected regression properties.

Required property	Test
Sandbox inventory cannot satisfy host measurement	<code>test_sandboxed_inventory_is_inadmissible</code> , <code>test_a_sandbox_cannot_manufacture_a_pass</code>
Dropping measurement_context fails closed	<code>test_contextless_inventory_is_inadmissible</code>
Partial inventories cannot improve a verdict	<code>test_partial_inventory_cannot_improve_a_verdict</code>
Coverage cannot be self-declared	<code>test_coverage_is_derived_from_paths_not_self_declared</code>
Renamed unresolved states decide identically	<code>test_renaming_unknown_to_operator_evidence_does_not_move_the_verdict</code>
Repeated runs produce identical graph and digest	<code>test_consecutive_runs_share_an_evidence_digest</code> , <code>test_graph_is_a_pure_function_of_its_inputs</code>
Evidence digest covers the inventory	<code>test_evidence_digest_covers_the_privilege_inventory</code>
Unresolved can never be verified	<code>test_unknown_can_never_be_verified</code> (2 sources × 5 paths)

The last row is the executable form of Theorem 1 restricted to `op1`; `op2` is covered by the coverage tests and `op4` by the context tests. No test currently exercises `op3` against a live host, because introducing a root-equivalent carrier would require mutating the host.

14. Threats to validity and expected reviewer questions

14.1 Threats to validity

- **n = 1.** A single Fedora host, one agent runtime. Distributions differ in sudoers defaults, polkit administrator identities, and which binaries ship setuid or with capabilities. The *method* generalises; none of the counts do. No claim in this paper is stated as a population property.
- **Discriminator judgement.** Several `NON_ROOT_EQUIVALENT` classifications rest on a bound that is documented behaviour plus verified package provenance (for example, that `at` executes jobs as the submitting user). These are stronger than assertion and weaker than proof.
- **Self-reported evaluation.** The findings were discovered by the system's own gates and evaluated by its authors. No independent red team has attempted to produce a false `VERIFIED_EXCLUSIVE_AUTHORITY`. §14.3 specifies the protocol that would; it has not been run.
- **Repeatability window.** Four consecutive runs is the system's own threshold, not a derived one.

14.2 Expected reviewer questions

Four objections are anticipated. Each is stated in the form we expect it, and answered with the position the paper actually holds rather than a stronger one.

Q1. Is this just Docker group auditing?

No. Docker is one measured carrier among 57 enumerated paths, and the fact that membership of the `docker` group confers host root is prior work, not a finding of this paper (§2, §12.1). Removing Docker from the host does not produce a positive verdict. Computed by running the model with both carriers reclassified and every required surface still covered, the verdict becomes `BLOCKED_PRIVILEGE_UNCERTAIN` with specific reason `BLOCKED_PRIVILEGE_UNCERTAIN_UNKNOWN_PATHS`: 18 operator-evidence paths remain across `sudo`, `polkit`, `groups`, `setuid` and `systemd`, and one registry `UNKNOWN` (`sudo_polkit_interactive`) remains alongside them. The specific reason names the `UNKNOWN` class rather than the operator-evidence class because the derivation reports the weaker evidential state first — a path nobody has characterised outranks a path whose deciding command is known but unreadable. The contribution is the verification model that produces that second verdict rather than a pass — a fail-closed decision procedure with a proved safety property (§9), a coverage gate that cannot be satisfied by assertion (§7.3), and an admissibility predicate that rejects an inventory which cannot prove where it was measured (§7.2). A Docker-specific audit would have reported the same carrier and nothing else; it would not have detected any of the three findings in §12, none of which concern Docker.

Q2. Is a single-host evaluation enough?

Not for a general claim, and no general claim is made. The evaluation is $n = 1$ and every count in §13 is a property of the evaluated host; §14.1 states this first among the threats to validity, and no claim in the paper is phrased as a population property. What is offered as generalising is the *method* — the three-valued model, the admissibility predicate, the derived-coverage gate, and the recorded-versus-consumed failure class — together with the observation that this class was detectable at all only because the mechanisms were in place. The findings themselves are structural rather than incidental: `setuid` enumeration cannot discover file capabilities on any Linux host, not merely on this one (§12.2). §17.3 names the cross-distribution study that would bound which results are host-specific, and it has not been run.

Q3. Can the verifier verify itself?

No, and the paper does not claim it can. §16.3 states the boundary flatly: the verifier is evidence-producing software, not a formally verified oracle. Nothing currently checks that the implemented checks are the checks the governance requirement intended, and a regression suite written by the same author as the code is not independent evidence of that correspondence — which is precisely how the original `unknown_paths` defect survived review (§1).

Two consequences are disclosed rather than deferred. First, three gaps are open at submission time: the unenforced gate-to-verifier coupling, the specified but unexercised set *Q*, and — found while formalising for this revision — that the blocking path sets are read from fields the inventory *publishes* rather than derived from its per-path classifications (§9.6, §16.3). The third is the paper's own §12.5 rule applied to the paper's own model, and it is not yet fixed. Second, the meta-verifier that would close the gap is specified in §17.2 as future work, with its first clause — every governance requirement must have an executable check that decides it — being the clause that would have caught the original defect.

Q4. Is the surface list complete?

No. `REQUIRED_SURFACES` contains seven entries, and the coverage gate proves coverage *of that list*, not of the operating system. Finding 1 is empirical proof that such a list can be wrong, since the list was believed complete until a surface with no discovery method at all was found in it (§12.2). §16.1 names seven further surfaces known to be absent, each capable of conferring authority.

The paper's position is that completeness is meaningful only relative to a *registered* discovery surface, and that the registration must be explicit enough to be reviewable. §17.1 specifies the Authority Surface Registry in which each surface carries its own `discovery_method`, so that a surface with none is a declared blind spot that can be made to block rather than an implicit property of a collector. That registry is proposed and not built; until it exists, the completeness claim in §6.1 is explicitly relative and §16.1 is the honest statement of what it excludes.

14.3 Proposed independent adversarial evaluation

The evaluation in §13.1–§13.3 is a measurement of a host, not an adversarial test of the verifier. What follows is a protocol we propose and have **not executed**; it is stated at this level of detail so that a third party can run it without our participation, and so that the claims of §13.3 can be falsified by someone with an incentive to falsify them.

Attacker model. The red team is given the verifier source, the evidence schema, and the full artifact set (consistent with T6 — none of this is secret). It may influence host state within the agent's own authority, and may supply inventories, contexts and artifacts to the verifier. It may not modify the verifier source, hold root, or edit artifacts after the digest is computed; those are the host administrator's powers, and an adversarial administrator is out of scope (§4).

Success condition (single, binary). A **false positive** is any input under which the system emits `VERIFIED_EXCLUSIVE_AUTHORITY` / `authority_exclusivity_proven: true` while a reachable authority carrier exists on the host at the moment of measurement. Anything else — a degraded blocker, a crash, a hang — is a lesser finding and is reported separately.

Table 14: Attack goals and the mechanism each targets.

#	Attacker goal	Targets	Predicted outcome under the current model
G1	Hide a privilege carrier from the enumerator	The curated <code>REQUIRED_SURFACES</code> list (§16.1)	Expected to succeed for any surface not on the list; this is the open limitation, and the finding of interest is <i>which</i> surface

#	Attacker goal	Targets	Predicted outcome under the current model
G2	Submit an incomplete inventory	Clause (ii), coverage derived from path ids (§7.3)	Blocked by Theorem 1/op2 — but see C2 (§9.5): the blocker degrades to a generic one
G3	Manipulate the context fingerprint	<code>admissible()</code> (§7.2), clause (i)	Blocked for the four measured conjuncts; the open question is whether a context exists that satisfies all four <i>and</i> suppresses a path
G4	Achieve digest stability without measurement completeness	Layer 3 whitelist (§10)	Blocked for inventory content post-Finding 2; untested for evidence <i>outside</i> the whitelist
G5	Exploit verifier/model mismatch	The gap between <code>exclusivity_model.py</code> and <code>verify_v3.py</code> (§11, dispatcher tests)	Unknown. The dispatcher tests assert the model is called; they do not assert the verifier honours the returned verdict in every branch
G6	Exploit the specified-and-unexercised set Q (§9.6)	Branch 2 of the verdict function	Unknown; no discriminator populates Q , so the branch is reachable only through graph closure

G1 and G5 are the goals we expect to be productive, and G5 is the one the authors are least able to test honestly, since the same author wrote both sides of the interface. G3 is the goal whose failure would most strengthen the paper.

Reporting. Each attempt should record the input, the emitted verdict and specific reason, the evidence digest, and whether a carrier was in fact reachable — the last established independently of the verifier, by exercising the carrier on a disposable host.

15. Reproducibility

```
python3 privilege_context.py      # rc=0 only in a host-representative context
python3 privilege_topology.py    # raw inventory, 7 surfaces
python3 privilege_resolution.py  # terminal states; rc=2 if not host-representative
python3 operator_evidence.py    # deterministic evidence checklist
python3 verify_v3.py            # 19 conditions, verdict, evidence digest
python3 cutover_gate.py --write  # readiness gate, no container started
python3 cutover_decision.py     # final decision from bound inputs
python3 -m unittest discover -s tests
python3 cutover_gate_control.py
sha256sum -c sha256sums.final.txt
sha256sum -c sha256sums.paper.txt
```

Artifacts: `PRIVILEGE_TOPOLOGY_FINAL.json`, `OPERATOR_EVIDENCE_CHECKLIST.json`, `CUTOVER_DECISION.json`, `verification_result.json`, `PRIVILEGE_GRAPH.json`, `attack_results.json`, `sha256sums.final.txt`, `sha256sums.paper.txt`.

Table 16: Provenance of every numeric claim. Each value below was recomputed from the named artifact while preparing this revision; none is transcribed. A number appearing in the body of this paper that is not in this table is either a section, figure or table reference, or a value quoted from the command output reproduced in §13.3.

Claim	Value	Artifact	Field
Authority paths measured	57	<code>PRIV-ILEGE_TOPOLOGY_FINAL.json</code>	<code>paths cardinality</code>
ROOT_EQUIVALENT	2	<code>PRIV-ILEGE_TOPOLOGY_FINAL.json</code>	<code>paths[*].classification</code>

Claim	Value	Artifact	Field
NON_ROOT_EQUIVALENT	37	PRIV-ILEGE_TOPOLOGY_FINAL.json	paths[*].classification
RE-QUIRES_OPERATOR_EVIDENCE	18	PRIV-ILEGE_TOPOLOGY_FINAL.json	paths[*].classification
NOT_PRESENT	0	PRIV-ILEGE_TOPOLOGY_FINAL.json	paths[*].classification
Blocking paths	20	PRIV-ILEGE_TOPOLOGY_FINAL.json	counts.blocking
Surfaces enumerated	7	PRIV-ILEGE_TOPOLOGY_FINAL.json	distinct surface: prefixes in paths
File-capability paths	11	PRIV-ILEGE_TOPOLOGY_FINAL.json	filecaps:* paths
setuid paths	35	PRIV-ILEGE_TOPOLOGY_FINAL.json	setuid:* paths
setuid digest MATCH, bounded	18	PRIV-ILEGE_TOPOLOGY_FINAL.json	setuid:* evidence + classification
setuid digest MATCH, unresolved	6	PRIV-ILEGE_TOPOLOGY_FINAL.json	as above
setuid no match, bounded	6	PRIV-ILEGE_TOPOLOGY_FINAL.json	as above
setuid no match, unresolved	5	PRIV-ILEGE_TOPOLOGY_FINAL.json	as above
Measured as host identity	True	PRIV-ILEGE_TOPOLOGY_FINAL.json	measurement_context.host_representative
Conditions verified	19	verification_result.json	conditions cardinality
Conditions met	14	verification_result.json	conditions[*].met is true
Operator-evidence items	18	OPERATOR_EVIDENCE_CHECKLIST.json	counts.total
...requiring privileged access	15	OPERATOR_EVIDENCE_CHECKLIST.json	counts.requires_privileged_access

Claim	Value	Artifact	Field
...requiring analysis only	3	OPERAT-OR_EVIDENCE_CHECK-LIST.json	counts.analysis_only_no_privilege_needed
authority_exclusivity_proven	False	CUTOVER_DECISION.json	authority_exclusivity_proven
Closure gates evaluated	8	CUTOVER_DECISION.json	closure gate map
Runs recorded	19	run_history.jsonl	line count
Distinct digests over those runs	3	run_history.jsonl	evidence_digest
Final four runs identical	True	run_history.jsonl	last four evidence_digest
Evidence digest	ab4ae6b-cb450971214b4e4e6a34be75147653826400ccd09c5e3786415e1c9ac	verification_result.json	evidence_digest
Verifier lines of Python	6788	working tree	collectors, model, verifier, gates, tests
...of which tests	1028	working tree	tests/*.py
Runtime lines under verification	1580	working tree	the V3 system itself, excluded from the figure above

Two rows deserve comment. `Runs recorded` counts every recorded run, not only the four that satisfy condition 18; the three distinct digests across those runs are the pre-correction digest, the post-`filecaps` digest, and the current one, which is why the repeatability count was reset twice (§12.3). `Verifier lines of Python` deliberately excludes the 1,580 lines of the V3 runtime *being* verified: counting the subject of the measurement inside the size of the measuring apparatus would overstate the latter.

16. Limitations

16.1 The surface list is curated

`REQUIRED_SURFACES` contains seven entries. The coverage gate proves coverage of that list, and Finding 1 demonstrated empirically that the list can be wrong.

Table 15: Known omitted surfaces.

Omitted surface	Why it matters
Kernel keyrings	Key material and delegated access outside the filesystem
systemd <i>user</i> units and generators	User-controlled units can execute at login or boot
cron and <code>at</code> execution chains	Spool directory writability and PAM environment
Runtime injection paths	<code>LD_PRELOAD</code> , <code>LD_AUDIT</code> , <code>ptrace</code> against processes of other principals
Container image and registry supply chain	Content executed by an already-authorized runtime
Filesystem ACLs beyond the classical mode bits	An ACL can replace group membership as a grant [acl5]
Writable service unit <code>EnvironmentFile</code> / <code>drop-ins</code>	Indirect control of a root unit

Until a discovery mechanism enumerates these, the completeness claim is explicitly relative, and this limitation is the honest reading of §6.1's "relative to the surface list".

16.2 Operator evidence cannot be closed by the agent

15 of 18 items require reads the agent cannot perform. This is a property of the threat model (T7), not a defect, but it means the system's terminal state on this host is *blocked pending a human*, and no engineering inside the agent's authority can change that.

16.3 Verifier integrity boundary

The verifier is currently evidence-producing software, not a formally verified oracle. It emits artifacts a reader can check against each other and against the host; it does not emit a proof, and nothing checks the verifier itself. This is the precise boundary:

- **What is bound.** Inputs are digested and recorded (§10, Layer 1); manifests are append-only (Layer 2); the evidence digest is sensitive to the inventory after Finding 2 (Layer 3); coverage, the privileged-access flag and the specific reason are derived rather than declared (§11).
- **What is not bound.** That the implemented checks are the checks the governance requirement intended. A requirement never translated into an executable check is invisible both to the code and to a regression suite written by the same author — which is exactly how the original `unknown_paths` defect survived review.
- **A specific disclosed gap.** The gate-to-verifier coupling for the readiness checklist is disclosed rather than enforced: `verify_v3.py` contains no reference to `cutover_gate.py`, so a green verifier verdict is closure only if the gate also cleared against the same host state.
- **A second disclosed gap.** `Q (AUTHORITY_EQUIVALENT)` is specified but has no producer (§9.6), so branch 2 of the verdict function is only partially exercised.
- **A third disclosed gap.** The blocking path sets `R`, `Q` and `U` are read from fields the inventory publishes, not derived from its per-path classifications (§9.6, item 4). Coverage was moved from declared to derived after Finding 1; these were not. The paper's own rule (§12.5) says they should be.

Contrast with artifacts that *are* verified end to end — a proof-carrying mechanism where the consumer checks a witness [Nec97, MMNS11], or a machine-checked kernel or compiler [Kle09, Ler09]. This system is not in that class and should not be read as being in it. §17.2 states what closing the gap would require.

16.4 Scope of the security claim

The system verifies that a specific agent identity cannot reach a specific authority principal on a specific host at a specific time. It does not verify the broker's request validation logic, the cryptographic properties of receipts, or resistance to a compromised authority principal. Nor does it address the input channel: an agent that can be induced to make a harmful *request* [GAM+23, Deb24] is a separate problem addressed by separate mechanisms [Deb25].

17. Future work

17.1 Authority Surface Registry

Replace the curated constant with a first-class registry in which each surface carries its own discovery and measurement method, so that "how would we find this class of thing" becomes reviewable data rather than an implicit property of a collector.

surface_id	stable identifier, e.g. "filecaps"
discovery_method	how instances are found (getcap -r; find -perm -4000; xattr sweep; parse unit files) – the field Finding 1 shows must be explicit, since a surface with no discovery method is unmeasurable by construction
measurement_method	what is read per instance in order to classify it
classification	terminal state assigned
confidence	measured documented-behaviour inferred
review_status	unreviewed reviewed contested

Three properties follow. A surface present in the registry with no `discovery_method` is a declared blind spot and can be made to block. `confidence` separates classifications resting on measurement from those resting on documented behaviour (§14.1), which is currently visible only by reading each discriminator. And a registry makes a surface's *disappearance* distinguishable from a surface that was never declared — which is the missing ingredient in counterexample C2 (§9.5), where deleting an inventory's only `container_runtimes` path currently degrades a named blocker into a generic one.

17.2 Verifier Integrity Layer

A meta-verifier whose subject is the verifier:

```

for each governance requirement R:
    assert ∃ executable check C : C decides R           # no unimplemented policy
for each verdict V emitted:
    assert ∃ provenance chain from V to measured evidence
for each state S reachable in the model:
    assert S ∈ fail-closed set ∨ S is justified by measurement
assert verifier logic ≡ governance requirements      # by review, then by test

```

The first clause is the one that would have caught the original defect: the requirement "no unresolved privilege path may certify" existed in the design and had no executable check. The second is a requirement-to-test mapping in the direction the field usually neglects — not "does this test pass" but "which requirement does this test discharge, and which requirement discharges none". The last clause is not fully automatable and is stated as such.

17.3 Other directions

- **Independent adversarial evaluation.** The protocol in §14.3, executed by a team with an incentive to produce a false `VERIFIED_EXCLUSIVE_AUTHORITY`.
- **Cross-distribution study.** Repeating the measurement across distributions would establish which findings are host-specific and which are structural, and would put a bound on the $n = 1$ limitation.
- **Continuous re-measurement.** The verdict is a statement about a moment. A host that grows a new setuid binary after certification is uncertified, and nothing currently notices.
- **Formalisation.** §9 is a hand proof over a small verdict function. It is small enough to admit a machine-checked version, which would also settle whether the label overloading of §9.5 can be removed without weakening the precedence rationale of §8.2.

18. Conclusion

We described an evidence-backed authority verification system for an autonomous agent deployment and reported its terminal verdict on the evaluated host: `BLOCKED_ROOT_EQUIVALENCE`, specific reason `BLOCKED_ROOT_EQUIVALENCE_DOCKER`, `authority_exclusivity_proven: false`. Two root-equivalent car-

riers remain measured and present, and 18 further paths remain unresolved because the deciding artifacts are unreadable without privileged access.

The architectural claim is narrow and, we believe, correct: **the system fails closed when authority exclusivity cannot be proven**. It certifies nothing while any path is unresolved; it treats an inventory that cannot prove where it was measured as absent rather than clean; and §9 gives the property a proof — no unresolved path, no missing surface, no context-invalid inventory, and no added root-equivalent carrier can yield a positive verdict. The same section records what is *not* true: the verdict labels are not a severity total order, and an adversary who can delete evidence can degrade a specific blocker into a generic one without ever approaching a certification.

The findings we consider most useful are the two failures of the verifier itself. Both are instances of one pattern: a fact was recorded, and a different fact was consumed. Setuid scanning cannot see file capabilities, so a whole surface was missing while coverage reported complete; and an evidence digest that omitted the inventory reported four identical runs across a change of eleven paths. Neither was found by an adversary. Both were found by gates built to enforce the distinction between measured absence and unresolved possibility — which is the argument for building the distinction even when, and especially when, the system it will be turned against is one's own.

The general claim is the one we would defend beyond this host: a verification system is a governed component, not a trusted one. It must produce evidence about itself of the same kind it demands of its subject, and it must never be permitted to convert an incomplete measurement into a safety certification. That the system's conclusion here is "blocked, and here is precisely what is unresolved and who can resolve it", rather than a certification, is the correct outcome for a verifier that cannot prove what it is being asked to certify.

19. References

Every entry below supports a specific claim in the text. Primary-source documentation is cited in preference to secondary description wherever the mechanism itself is the claim.

Linux mechanism (primary sources).

- [cap7] *capabilities(7)*, Linux man-pages project. <https://man7.org/linux/man-pages/man7/capabilities.7.html> — file capability sets, the permitted/effective/inheritable distinction, and the `=ep / =ei` semantics used in §12.2.
- [cred7] *credentials(7)*, Linux man-pages project. <https://man7.org/linux/man-pages/man7/credentials.7.html> — process credentials and supplementary group sets (§6.3, §7.2).
- [users7] *user_namespaces(7)*, Linux man-pages project. https://man7.org/linux/man-pages/man7/user_namespaces.7.html — uid/gid map semantics underlying the admissibility predicate (§7.2).
- [nnp] *No New Privileges flag*, Linux kernel documentation (`Documentation/userspace-api/no_new_privs.rst`); see also *prctl(2)*, `PR_SET_NO_NEW_PRIVS` — why `NoNewPrivs = 1` suppresses every setuid transition (§7.2).
- [unix7] *unix(7)*, Linux man-pages project — permission checks on `connect(2)` to an `AF_UNIX` socket (§12.1).
- [acl5] *acl(5)* and *setfacl(1)*, Linux man-pages project — ACL entries as an alternative grant to group membership (§12.1, §16.1).
- [sudoers5] *sudoers(5)*, Todd C. Miller — policy semantics and the non-interactive `sudo -n -l` query (§6.3).

- [libcap] *getcap(8)*, *setcap(8)* and the libcap documentation — file capability storage in extended attributes (§6.3, §12.2).
- [ns7] *namespaces(7)*, Linux man-pages project. <https://man7.org/linux/man-pages/man7/namespaces.7.html> — the confinement primitive discussed in §2 and the mechanism behind Finding 3.
- [seccomp2] *seccomp(2)*, Linux man-pages project. <https://man7.org/linux/man-pages/man2/sec-comp.2.html> — syscall filtering; a `Seccomp` mode other than 0 is a disqualifier in the admissibility predicate (§7.2).
- [landlock] *Landlock: unprivileged access control*, Linux kernel documentation (`Documentation/user-space-api/landlock.rst`) — unprivileged filesystem confinement, cited in §2.

Sandboxing and confinement.

- [gvisor] E. G. Young, P. Zhu, T. Caraza-Harter, A. C. Arpaci-Dusseau, and R. H. Arpaci-Dusseau. *The True Cost of Containing: A gVisor Case Study*. In Proceedings of the 11th USENIX Workshop on Hot Topics in Cloud Computing (HotCloud '19), 2019 — a user-space kernel interposing on the syscall interface (§2).
- [firecracker] A. Agache, M. Brooker, A. Iordache, A. Liguori, R. Neugebauer, P. Piwonka, and D.-M. Popa. *Firecracker: Lightweight Virtualization for Serverless Applications*. In Proceedings of the 17th USENIX Symposium on Networked Systems Design and Implementation (NSDI '20), pp. 419–434, 2020 — moving the isolation boundary below the kernel (§2).

Privilege enumeration tooling.

- [peass] C. Polop et al. *PEASS-ng: Privilege Escalation Awesome Scripts Suite (LinPEAS)*. <https://github.com/peass-ng/PEASS-ng> — breadth-first enumeration of Linux escalation candidates, ranked for a human reader (§2).
- [lynis] CISOfy. *Lynis: security auditing tool for Linux, macOS and UNIX-based systems*. <https://cisofy.com/lynis/> — host hardening audit (§2).
- [oscap] *OpenSCAP*. <https://www.open-scap.org/> — policy-baseline evaluation; its XCCDF rule-result vocabulary (`notchecked`, `notapplicable`, `unknown` alongside `pass` / `fail`) is the closest prior representation of a check that did not decide (§2).

Containers.

- [dockersec] *Docker Engine security*, Docker documentation. <https://docs.docker.com/engine/security/> — the "Docker daemon attack surface" section, quoted in §12.1.
- [rootless] *Run the Docker daemon as a non-root user (rootless mode)*, Docker documentation. <https://docs.docker.com/engine/security/rootless/> — the remediation named in §12.1.
- [Bui15] T. Bui. *Analysis of Docker Security*. arXiv preprint, 2015.
- [CMD16] T. Combe, A. Martin, and R. Di Pietro. *To Docker or Not to Docker: A Security Perspective*. IEEE Cloud Computing, 2016.
- [NCC16] A. Grattafiori et al. *Understanding and Hardening Linux Containers*. NCC Group whitepaper, 2016.
- [Lin18] X. Lin, L. Lei, Y. Wang, J. Jing, K. Sun, and Q. Zhou. *A Measurement Study on Linux Container Security: Attacks and Countermeasures*. In Proceedings of the 34th Annual Computer Security Applications Conference (ACSAC '18), pp. 418–429, 2018. DOI 10.1145/3274694.3274720.

Authorization and protection principles.

- [polkit] *polkit Reference Manual*, freedesktop.org. <https://www.freedesktop.org/software/polkit/docs/latest/> — the authorization architecture, rules precedence, and administrator identity resolution relied on in §6.3 and §7.4.
- [SS75] J. H. Saltzer and M. D. Schroeder. *The Protection of Information in Computer Systems*. Proceedings of the IEEE, 63(9):1278–1308, September 1975. DOI 10.1109/PROC.1975.9939 — fail-safe defaults and complete mediation, the principles §3 and §9 instantiate at verification time rather than at access time.
- [And72] J. P. Anderson. *Computer Security Technology Planning Study*. ESD-TR-73-51, U.S. Air Force Electronic Systems Division, 1972 — the reference-monitor arrangement the broker of §5 instantiates.
- [CWD02] H. Chen, D. Wagner, and D. Dean. *Setuid Demystified*. In Proceedings of the 11th USENIX Security Symposium, pp. 171–190, 2002 — the uid-setting semantics whose subtlety motivates measuring rather than assuming setuid behaviour (§6.3).
- [PFH03] N. Provos, M. Friedl, and P. Honeyman. *Preventing Privilege Escalation*. In Proceedings of the 12th USENIX Security Symposium, 2003 — the privilege-separation structure of §4.
- [HM08] S. E. Hallyn and A. G. Morgan. *Linux capabilities: making them work*. In Proceedings of the Linux Symposium, 2008 — the intent of file capabilities as a replacement for setuid (§12.2).

Verification, evidence, and provenance.

- [Liv15] B. Livshits, M. Sridharan, Y. Smaragdakis, O. Lhoták, J. N. Amaral, B.-Y. E. Chang, S. Z. Guyer, U. P. Khedker, A. Møller, and D. Vardoulakis. *In Defense of Soundness: A Manifesto*. Communications of the ACM, 58(2):44–46, 2015. DOI 10.1145/2644805 — the discipline of declaring what an analysis does not cover, applied in §1 and §16.1.
- [Nec97] G. C. Necula. *Proof-Carrying Code*. In Proceedings of the 24th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL '97), pp. 106–119, 1997. DOI 10.1145/263699.263712 — the standard this system does not meet, stated as the gap in §16.3.
- [MMNS11] R. M. McConnell, K. Mehlhorn, S. Näher, and P. Schweitzer. *Certifying algorithms*. Computer Science Review, 5(2):119–161, 2011 — output accompanied by a witness the consumer can check without trusting the producer; the design target of §9.
- [Kle09] G. Klein et al. *seL4: Formal Verification of an OS Kernel*. In Proceedings of the 22nd ACM Symposium on Operating Systems Principles (SOSP), 2009 — an artifact in the verified class this system is not in (§16.3).
- [Ler09] X. Leroy. *Formal verification of a realistic compiler*. Communications of the ACM, 2009 — as above, for a compiler (§16.3).
- [TA19] S. Torres-Arias, H. Afzali, T. K. Kuppusamy, R. Curtmola, and J. Cappel. *in-toto: Providing farm-to-table guarantees for bits and bytes*. In Proceedings of the 28th USENIX Security Symposium, pp. 1393–1410, 2019 — binding each step's evidence to the step that produced it (§10, Layer 1).
- [repro] *Reproducible Builds*. <https://reproducible-builds.org/> — the bit-identical-output property that §6.1 distinguishes from evidence integrity.
- [slsa] *SLSA: Supply-chain Levels for Software Artifacts*. <https://slsa.dev/> — provenance attestation levels, cited for the same distinction.

Autonomous agent security.

- [GAM+23] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz. *Not what you've signed up for: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection*. In Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec '23), 2023. arXiv:2302.12173 — the mechanism by which the adversary of §4 comes to execute under the agent's credential.
- [Deb24] E. Debenedetti et al. *AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents*. In Advances in Neural Information Processing Systems 37 (NeurIPS 2024), Datasets and Benchmarks Track, 2024. arXiv:2406.13352 — application-layer adversarial evaluation, the complement to the host-layer measurement of §12.
- [Deb25] E. Debenedetti, I. Shumailov, T. Fan, J. Hayes, N. Carlini, D. Fabian, C. Kern, C. Shi, A. Terzis, and F. Tramèr. *Defeating Prompt Injections by Design*. arXiv:2503.18813, 2025 — design-level containment of untrusted data flow, orthogonal to the authority question of §2.
- [Rua24] Y. Ruan et al. *Identifying the Risks of LM Agents with an LM-Emulated Sandbox*. In Proceedings of the International Conference on Learning Representations (ICLR), 2024 — risk identification by observation of emulated executions, an instance of the argument-from-observation critiqued in §1.